

# Verifiable Divide-and-Conquer: Proof-Merging Beyond the $\log n$ -Barrier

Omer Paneth<sup>\*1</sup> and Rafael Pass<sup>†1,2</sup>

<sup>1</sup>Tel Aviv University.

<sup>2</sup>Cornell Tech and Technion.

April 5, 2026

## Abstract

Succinct non-interactive arguments (SNARGs) for  $P$  are a powerful tool for verifiably outsourcing a computation to a single entity. In this work, we study verifiable outsourcing in a distributed divide-and-conquer setting: a party tasked with solving a large computational problem divides it into smaller subproblems, delegates each subproblem to a different party, and then combines their solutions. Moreover, each party may recursively apply the same strategy, further delegating its computation. Verifying such recursive outsourcing requires SNARGs that are *mergeable*, so that proofs for subproblems can be efficiently combined into a single proof for the original problem.

While mergeable proofs have been known for nearly two decades (either based on heuristic assumptions such as SNARKs or, more recently, on standard hardness assumptions), in all existing approaches security degrades exponentially with the number of recursive merges. In this work, we overcome this barrier. Assuming the Learning with Errors (LWE) assumption, we construct a mergeable SNARG for  $P$  that supports an unbounded polynomial number of recursive merges. The proof size grows only linearly with the depth of the merge tree, and is independent of its total size.

---

<sup>\*</sup>Omer Paneth is a member of the Checkpoint Institute of Information Security and is supported by Len Blavatnik and the Blavatnik Foundation. Supported in part by AFOSR Award FA9550-23-1-0312. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Government or AFOSR.

<sup>†</sup>Rafael Pass is supported in part by NSF Award CNS 2149305, AFOSR Award FA9550-23-1-0387, AFOSR Award FA9550-23-1-0312, AFOSR Award FA9550-24-1-0267, ISF Award 2338/23 and ERC Advanced Grant KolmoCrypt - 101142322. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Government, the AFOSR, the European Union or the European Research Council Executive Agency.

# Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                  | <b>3</b>  |
| 1.1      | Our Results . . . . .                | 4         |
| 1.2      | Technical Overview . . . . .         | 8         |
| <b>2</b> | <b>Preliminaries</b>                 | <b>18</b> |
| <b>3</b> | <b>Batch Arguments</b>               | <b>19</b> |
| <b>4</b> | <b>Mergeable Arguments</b>           | <b>20</b> |
| <b>5</b> | <b>Construction</b>                  | <b>22</b> |
| <b>6</b> | <b>Analysis</b>                      | <b>24</b> |
| 6.1      | Completeness. . . . .                | 24        |
| 6.2      | Efficiency. . . . .                  | 25        |
| 6.3      | Soundness. . . . .                   | 26        |
| <b>A</b> | <b>SNARGs with a Linear Verifier</b> | <b>34</b> |

# 1 Introduction

In verifiable computation, a prover performs a deterministic computation and seeks to convince a verifier of the correctness of the output by sending a single-message proof. For a computation running in time  $T$ , the prover constructs a proof in  $\text{poly}(T)$  time, while the verifier should run strictly faster than  $T$ —ideally in time  $n \cdot \text{polylog}(T)$ , where  $n$  denotes the input and output length.

Since the existence of such efficiently verifiable proofs with statistical soundness is subject to known barriers [GVW02], recent work has focused on *computationally sound* proofs. In this setting, the prover and verifier are given a public key of length  $\lambda$  (the security parameter), and both the running time and the proof length may depend on  $\text{poly}(\lambda)$ . In particular, the proof size is  $\text{poly}(\lambda, \log T)$  and the verification time is  $n \cdot \text{poly}(\lambda, \log T)$ . Soundness requires that no computationally bounded adversary (formally, a polynomial-size circuit) can produce a false statement together with an accepting proof, except with negligible probability in  $\lambda$  over the choice of the public key. Such proof systems are known as *succinct non-interactive arguments for deterministic computations* (*SNARGs for P*). In recent years, SNARGs for P have been constructed under a variety of standard computational assumptions [CJJ21, WW22, CGJ<sup>+</sup>23, KLVW23].

**The distributed setting: divide-and-conquer.** SNARGs for P are well suited to settings in which a single party performs the entire computation. However, they are less useful in *distributed* settings. For example, a common paradigm for solving large computational problems is *divide and conquer*: a party tasked with solving a problem partitions it into smaller subproblems, delegates each subproblem to a different party, and combines their solutions into a solution to the original problem. Moreover, each party may recursively apply the same strategy, further subdividing its task. For example, consider the problem of counting the number of primes in an interval  $[a, b]$ . This task can be divided into subproblems over intervals  $[a, c]$  and  $[c, b]$ , whose respective solution  $m_1$  and  $m_2$  can be combined into a solution  $m = m_1 + m_2$  for the original interval  $[a, b]$ .

The central question we address in this work is whether such distributed computations can still be verified efficiently. A naive approach is for the parties at the leaves of the delegation tree to construct proofs for their individual computations. Each internal node then collects the proofs from its children and forwards them upward, allowing the root to verify the entire computation by checking all the leaf proofs. The drawback of this approach is that it fails to preserve succinctness: both the proof size and the verification time grow linearly with the number of leaves in the delegation tree.

**Proof merging.** A powerful approach to addressing this challenge is *proof merging*: Suppose a problem  $x$  is divided into two subproblems  $x_1$  and  $x_2$ , with corresponding proofs  $\Pi_1$  and  $\Pi_2$ . Can we efficiently merge them into a single short proof for  $x$ , whose size is not much longer than  $\max(|\Pi_1|, |\Pi_2|)$ ? If such merging is possible, we can regain succinctness: instead of forwarding multiple proofs upward, each internal node merges its children’s proofs and passes on only the merged proof.

This idea was introduced by Valiant [Val08] and later extended through the notion of proof-carrying data (PCD) [CT10, BCCT13]. Valiant also proposed a simple method for merging proofs by providing a succinct proof of the existence of  $\Pi_1$  and  $\Pi_2$ . This approach requires the underlying proof system to be a *succinct non-interactive argument of knowledge for nondeterministic computations* (a *SNARK for NP*), where the knowledge property is formalized via an extractor.

However, this approach has two significant drawbacks. First, the security of existing SNARK constructions for NP is heuristic and can only be analyzed in idealized models (e.g., [Mic94]). Moreover, constructing SNARKs with explicit knowledge extractors faces strong barriers [BCPR16, BP15]. Second, recursive invocations of the extractor grow exponentially with the depth of the merge tree. As a result, the merged proofs are only conjectured to be sound for constant-depth trees. Even under the stronger assumption of SNARKs with *linear-time extractors*, the guarantee only extends to trees of depth  $O(\log \lambda)$ . A related line of work [BGH19, BCMS20] proposes more efficient proof merging via so-called folding schemes. Although these schemes are not SNARKs, they face the same barriers and are likewise restricted to  $O(\log \lambda)$  recursive merges due to recursive extraction.

More recently, [DGKV22, PP22] replaced SNARKs with a different proof system: *somewhere-extractable batch arguments for NP (BARGs)* [CJJ21]. Unlike SNARKs, BARGs can be constructed under standard assumptions [CJJ21, WW22, CGJ<sup>+</sup>23, KLVW23]. Moreover, BARG-based proof merging admits a constructive extractor with only additive polynomial overhead, suggesting that recursive extraction could, in principle, be carried out a polynomial number of times.

Nevertheless, even in this setting, the security proof breaks down after  $O(\log \lambda)$  recursive merges due to the BARG’s “somewhere extractability” guarantee. Roughly speaking, BARGs provide a weaker form of extractability in which only part of the witness can be extracted. In particular, after merging, one can extract either  $\Pi_1$  or  $\Pi_2$  but not both. Thus, to recursively extract a false proof, the reduction must correctly “guess” which branch to extract at each level; after  $\omega(\log \lambda)$  levels, the probability of success becomes negligible.

In summary, whether based on SNARKs for NP or on standard assumptions, all known proof-merging techniques encounter an apparent  $O(\log \lambda)$  barrier. This leads to the central question of this work:

*Can proof merging surpass the  $O(\log \lambda)$  barrier?  
Is verifiable divide-and-conquer possible for delegation trees of super-logarithmic depth?*

## 1.1 Our Results

In this work, we provide a positive answer to the above question under standard hardness assumptions.

**Verifiable divide-and-conquer.** We begin by formalizing a notion of mergeable SNARGs that enables efficient verification of distributed divide-and-conquer computations. For simplicity, we focus on decision problems specified by a language  $\mathcal{L}$ . Our treatment naturally extends to search problems by considering the language of instance–solution pairs.

We assume that  $\mathcal{L}$  can be decided by a deterministic algorithm, but we do not assume a fixed polynomial upper bound on its running time. Instead, we require the existence of an efficiently computable function  $T : \{0, 1\}^* \rightarrow \mathbb{N}$  such that  $x \in \mathcal{L}$  can be decided in deterministic time  $T(x)$ . For example, let  $\mathcal{L}_U$  denote the universal language consisting of all pairs  $(x, t)$  such that the Turing machine  $U$  accepts  $x$  within at most  $t$  steps. In this case, we may set  $T((x, t)) = t$ .

To capture general divide-and-conquer strategies, we do not restrict how problems are decomposed into subproblems. Instead, we abstract the process of merging solutions through the notion of an *instance merger*. An instance merger for  $\mathcal{L}$  is an algorithm  $\text{IM}$  that takes as input a sequence of instances  $x_1, \dots, x_k$  and outputs either a merged instance  $x$ , or  $\perp$  if the instances cannot be

merged. We require the merger to be sound: for any  $x_1, \dots, x_k \in \mathcal{L}$ , if  $\text{IM}(x_1, \dots, x_k)$  outputs  $x \neq \perp$ , then  $x \in \mathcal{L}$ . We further require that if  $x \neq \perp$ , then  $T(x_i) \leq T(x)$  for every  $i$ . In other words, merging instances does not make them easier to decide.

For example, consider the language  $\mathcal{L}_{\text{PRIME}}$  consisting of all tuples  $(a, b, m) \in \mathbb{N}^3$  such that there are exactly  $m$  primes in the interval  $[a, b]$ . A natural instance merger for  $\mathcal{L}_{\text{PRIME}}$  takes as input two instances  $(a, c, m_1)$  and  $(c', b, m_2)$ , and outputs  $(a, b, m_1 + m_2)$  if  $c = c'$ , and  $\perp$  otherwise. Indeed, if both  $(a, c, m_1)$  and  $(c, b, m_2)$  belong to  $\mathcal{L}_{\text{PRIME}}$  then also  $(a, b, m_1 + m_2) \in \mathcal{L}_{\text{PRIME}}$ .

We say that a SNARG for  $\mathcal{L}$  is *mergeable* (with respect to an instance merger  $\text{IM}$ ) if there exists a *proof merger* algorithm  $\text{PM}$  that takes as input a public key, instances  $x_1, \dots, x_k$ , and corresponding proofs  $\Pi_1, \dots, \Pi_k$ , and outputs a merged proof  $\Pi$ . The requirement is that if the SNARG verifier accepts each pair  $(x_i, \Pi_i)$  and the instance merger  $\text{IM}(x_1, \dots, x_k)$  outputs  $x \neq \perp$ , then the verifier also accepts  $(x, \Pi)$ . We emphasize that the soundness guarantee remains unchanged: given the public key, no polynomial-size adversary can produce an instance  $x \notin \mathcal{L}$  together with an accepting proof  $\Pi$  except with negligible probability.

In addition, the proof merger must satisfy a nontrivial efficiency requirement. Its running time should be polynomial in the size of the input instances and proofs, and must not grow with the decision time  $T(x)$  (otherwise, the merger could simply construct a proof for  $x$  from scratch). Furthermore, the merged proof should be significantly shorter than the concatenation of the  $k$  input proofs, and verifying it should be faster than verifying each proof individually. Importantly, to verify the merged proof  $\Pi$ , the verifier is given only the merged instance  $x$  and does not need to know the original instances  $x_1, \dots, x_k$  or their proofs.

**Main results.** We construct mergeable SNARGs for every language  $\mathcal{L}$  and every instance merger  $\text{IM}$  (subject to a mild condition specified below), assuming the existence of BARGs with certain properties that can be instantiated under the Learning with Errors (LWE) assumption.

**Theorem 1.1** (Informally-stated). *Fix a language  $\mathcal{L}$  as above and let  $\text{IM}$  be a “natural” instance merger for  $\mathcal{L}$ . Assuming the Learning with Errors assumption holds against polynomial-size attackers, there exists a mergeable SNARG for  $\mathcal{L}$  with the following efficiency: for instances  $x_1, \dots, x_k$  and their corresponding proofs  $\Pi_1, \dots, \Pi_k$ , the length of a merged proof is:*

$$\max |\Pi_i| + |(x_1, \dots, x_k)| \cdot \text{poly}(\lambda) \ .$$

Note that in the natural special case where the total length of the instances merged at each step is bounded by a fixed polynomial in  $\lambda$ , our construction supports merge trees of arbitrary polynomial size. In this case, the length of the merged proof simplifies to  $\max |\Pi_i| + \text{poly}(\lambda)$ .

**Tree-bounded instance mergers.** Our soundness definition restricts the adversary to producing an instance  $x$  such that  $T(x)$  is bounded by some polynomial in  $\lambda$ . This polynomial is fixed only in the security proof and does not affect the construction of the proof system itself. This restriction is common to all prior work constructing SNARGs for  $\text{P}$  from standard assumptions (even without proof merging), with the exception of [BPSS23], which achieves the stronger notion of universal arguments [BG08] under additional assumptions.

In the context of mergeable SNARGs, we further restrict soundness by requiring that the size of the merge tree for  $x$  is also polynomially bounded. Formalizing this restriction requires care, since the adversary does not explicitly specify the merge tree. Instead, we bound the size of the

merge tree by imposing a structural restriction on the instance merger itself. Intuitively, we require that there exists an efficiently computable bound on the size of *any* merge tree producing a given instance.

More formally, we say that an instance merger  $\text{IM}$  is *tree-bounded* if there exists an efficiently computable function  $W : \{0, 1\}^* \rightarrow \mathbb{N}^+$  such that for any  $x_1, \dots, x_k \in \{0, 1\}^*$ , if  $\text{IM}(x_1, \dots, x_k)$  outputs  $x \neq \perp$ , then  $W(x) \geq \sum W(x_i)$ . Our soundness definition restricts the adversary to producing an instance  $x$  for which both  $T(x)$  and  $W(x)$  are bounded by  $\text{poly}(\lambda)$ . In natural divide-and-conquer settings, the instance merger is typically tree-bounded. For example, for the language  $\mathcal{L}_{\text{PRIME}}$  with the merger described above, we may define  $W((a, b, m)) = b - a$ , the size of the interval.

Moreover, any language  $\mathcal{L}$  and instance merger  $\text{IM}$  can be augmented to become tree-bounded by attaching an explicit bound  $n$  to each instance. Specifically, define the augmented language  $\mathcal{L}' = \{(x, n) : x \in \mathcal{L}\}$  and the augmented instance merger:

$$\text{IM}'((x_1, n_1), \dots, (x_k, n_k)) = \left( \text{IM}(x_1, \dots, x_k), \sum n_i \right) .$$

Using a mergeable SNARG for  $\mathcal{L}'$ , a proof for an instance  $(x, n) \in \mathcal{L}'$  can only be constructed via merge trees of total size at most  $n$ , even if the original merger  $\text{IM}$  allows  $x$  to be derived through larger merge trees.

An important limitation of tree-bounded mergers is that they may exclude merge strategies that follow general directed acyclic graphs (DAGs) rather than trees. The reason is that an instance obtained via a polynomial-size merge DAG may also be derived by an exponential-size merge tree. We note that this restriction to tree-bounded mergers appears to be necessary even in very strong idealized models that allow straight-line extraction with only additive overhead [CGSY24].

**Shrinking merged proofs.** As noted above, Theorem 1.1 gives merged proof that grow only with the depth of the merge tree and not with its size, provided that the total length of the instances merged at each step is bounded by a fixed polynomial in  $\lambda$ . In some settings, however, the size of the merged instances may not be a priori bounded and may grow with the overall size of the merge tree.

For example, consider the conjunction language  $\mathcal{L}_{\text{CON}}$  consisting of all tuples  $(y_1, \dots, y_k)$  such that  $y_i \in \mathcal{L}'$  for every  $i \in [k]$ , where  $\mathcal{L}' \in \mathcal{P}$ . (This language can be viewed as a deterministic analogue of BARGs.) The natural instance merger for  $\mathcal{L}_{\text{CON}}$  takes two tuples and concatenates them into a single tuple.

For languages such as  $\mathcal{L}_{\text{CON}}$ , where merged instances grow with the size of the merge tree, the verification necessarily grows with the instance size. Nevertheless, we would still like the merged proofs themselves to grow only with the depth of the merge tree. We present a simple extension of Theorem 1.1 that achieve this. Roughly speaking, if the instances  $x_1, \dots, x_k$  can be reconstructed from the merged instance  $x = \text{IM}(x_1, \dots, x_k)$ , possibly using short advice, then the length of the merged proof can be made independent of the instance size. Equivalently, the proof length depends only on the *conditional time-bounded Kolmogorov complexity*  $cK^t(x_1, \dots, x_k \mid x)$  for any polynomial bound  $t$  [ZL70, LM91, LP22].

More formally, fix a polynomial-time reconstruction algorithm  $R$  such that for every  $x_1, \dots, x_k$  with  $\text{IM}(x_1, \dots, x_k) = x \neq \perp$ , there exists advice  $z \in \{0, 1\}^*$  satisfying  $R(x, z) = (x_1, \dots, x_k)$ . Then, given instances  $x_1, \dots, x_k$ , advice  $z$  and proofs  $\Pi_1, \dots, \Pi_k$ , the length of the merged proof is

$$\max |\Pi_i| + |z| \cdot \text{poly}(\lambda) .$$

For example, in the case of the language  $\mathcal{L}_{\text{CON}}$ , when two tuples  $x_1$  and  $x_2$  are merged by concatenation into a single tuple  $x$ , we can reconstruct  $x_1$  and  $x_2$  from  $x$  given only the length of  $x_1$  as advice. Consequently, the merged proofs grow only with the depth of the merge tree.

We note that for the special case of  $\mathcal{L}_{\text{CON}}$ , a related solution follows from the notion of multi-hop BARGs introduced in [DGKV22]. However, their construction guarantees soundness only when the merge tree structure is fixed non-adaptively (i.e., it does not depend on the public key), and it requires the verifier to know the structure of the merge tree rather than only the merged instance.

We conclude by mentioning another application of mergeable SNARGs with proof shrinking. Consider a distributed computation over a tree in which each leaf node is assigned an input label, and the label of every internal node is computed from the labels of its children. Such computations can be executed verifiably using mergeable SNARGs for a language  $\mathcal{L}_{\text{TR}}$ , where each instance corresponds to a node  $v$  and includes the labels of  $v$  and all nodes in its subtree. We merge  $k$  instances by attaching the roots of the corresponding subtrees to a new root and computing the new root's label. Combining Theorem 1.1 with our proof-shrinking technique yields a mergeable SNARG for  $\mathcal{L}_{\text{TR}}$  whose proof size grows only with the depth of the tree.

**Proof-carrying data.** A closely related line of work is proof-carrying data (PCD) [CT10]. In PCD, a distributed computation takes place over a directed acyclic graph (DAG). Each edge is labeled with some data, and each node uses the data on its incoming edges together with some local input to compute the labels of its outgoing edges. The data on each edge is accompanied by a proof certifying that the local computation performed by the node, as well as all prior computations in the DAG satisfy a prescribed compliance predicate. These proofs are required to remain succinct, independent of the size of the graph and the running time of the local computations.

PCDs are expressive enough to capture mergeable SNARGs: each edge can be labeled by an instance. A source node is compliant if each of its outgoing edges is labeled by an instance in  $\mathcal{L}$ . An internal node is compliant if the instance on each of its outgoing edges is obtained by merging the instances on its incoming edges using the instance merger.

In fact, PCDs are more general than mergeable SNARGs, as they can also handle nondeterministic languages. This is captured by the fact that the PCD verifier is not given the local input of each node (which can be viewed as an NP witness). However, even assuming SNARKs for NP, existing PCD constructions are known only for constant-depth graphs [BCCT13]. In the other direction, mergeable SNARGs for the language  $\mathcal{L}_{\text{TR}}$  described above can be viewed as a deterministic analogue of PCD for *deterministic* computations over *trees* with unbounded depth under standard assumptions.

**On PCDs for NP.** Although our focus is on mergeable SNARGs for deterministic languages, our results naturally extend to languages in NP. In this setting, the prover uses the NP witness directly, rather than simulating the deterministic decision procedure running in time  $T$ . As a result, the proof size grows with the size of the witness, but remains independent of the size of the merge tree.

Moreover, since our security reduction runs in time  $\text{poly}(T)$ , we rely on sub-exponential assumptions and set the security parameter to grow with the witness size. The key difference between such mergeable SNARGs for NP and the standard notion of PCD in the literature is that PCD typically considers a universal NP language, where the witness sizes are not bounded a priori and may grow with the merge tree. In contrast, our construction can be viewed as PCD for NP with an a priori

bound on the witness length.

## 1.2 Technical Overview

Our starting point is a simple mergeable proof system based on [DGKV22, PP22], which is limited to constant-depth merge trees. We first describe this construction and outline its security proof.

**Batch arguments for NP.** The main building block in the constructions of [DGKV22, PP22] is a somewhere-extractable batch argument for NP (BARG) [CJJ21]. A BARG is a non-interactive argument system for conjunctions of NP statements. The common input to the BARG prover and verifier is a list of instances  $x_1, \dots, x_k$ , and the prover is additionally given the corresponding witnesses of length  $m$ . For  $k \leq 2^\lambda$ , the proof has size  $\text{poly}(m, \lambda)$ , independent of  $k$ . The BARG is said to have *rate 1* if the proof size is  $m(1 + o(1))$ .

The knowledge-extraction property of a BARG consists of two components:

**Key indistinguishability:** For every index  $i \in [k]$ , it is possible to generate a public key  $\beta_i$  that is “programmed” on  $i$  and is computationally indistinguishable from an honestly generated key. The programmed key is generated together with a trapdoor  $\tau$  that enables extraction.

**Somewhere extraction:** Given a programmed key  $\beta_i$ , if an efficient adversary outputs instances  $x_1, \dots, x_k$  together with an accepting proof  $\Pi$  under  $\beta_i$ , then using the trapdoor  $\tau$  one can efficiently extract from  $\Pi$  a valid witness for  $x_i$ , except with negligible probability.

**Constant-depth mergeable proofs.** Fix a deterministic language  $\mathcal{L}$  equipped with an instance merger  $\text{IM}$ . Assuming the existence of BARGs, it is known how to construct a SNARG for  $\mathcal{L}$  [CJJ21]. We sketch how to use BARGs to transform any SNARG for  $\mathcal{L}$  into a mergeable SNARG supporting merge trees of constant depth, following the approach of [DGKV22, PP22].

The public key of the mergeable SNARG consists of the public key of the underlying SNARG for  $\mathcal{L}$  together with a public key for a BARG. The proof merger receives instances  $x_1, \dots, x_k$  and corresponding proofs  $\Pi_1, \dots, \Pi_k$ . It constructs a single BARG proof  $\tilde{\Pi}$  asserting that each instance  $x_1, \dots, x_k$  admits an accepting SNARG proof, using  $\Pi_1, \dots, \Pi_k$  as witnesses. It outputs a merged proof  $\Pi$  that contains the original instances  $x_1, \dots, x_k$  together with the BARG proof  $\tilde{\Pi}$ .

To verify a merged proof  $\Pi$  for an instance  $x$ , the verifier checks that  $\text{IM}(x_1, \dots, x_k) = x$  and that  $\tilde{\Pi}$  is accepting.<sup>1</sup> To support recursive merging, the public key includes a separate BARG public key for each level of the merge tree, so that at each level the merged proof either certifies BARG proofs from the level below or proofs of the underlying SNARG.

If each proof  $\Pi_i$  has length  $m$ , then the merged proof has length  $|(x_1, \dots, x_k)| + \text{poly}(m, \lambda)$ . While this bound is already nontrivial (assuming  $|x_i| \ll m$ ), a merge tree of super-constant depth would lead to proofs of super-polynomial size. Using rate-1 BARGs ensures that the proof length grows only linearly with the depth of the merge tree. However, as we explain next, the soundness analysis still limits the construction to constant-depth merge trees.

To argue soundness, suppose there exists an adversary  $\text{Adv}$  that produces a false statement  $x \notin \mathcal{L}$  with  $\text{T}(x) \leq \text{poly}(\lambda)$ , together with an accepting proof  $\Pi$ , with non-negligible probability  $\epsilon$ . If  $\Pi$  is not a merged proof, then we immediately violate the soundness of the underlying

<sup>1</sup>Although  $\text{IM}$  may not run in linear time, a linear-time verifier can be obtained by shifting the verifier’s work to the prover/proof merger using RAM SNARGs, which can also be constructed from BARGs. See Appendix A for details.

SNARG. Otherwise,  $\Pi$  consists of instances  $x_1, \dots, x_k$  and a BARG proof  $\tilde{\Pi}$ . If  $\Pi$  is accepting, then  $\text{IM}(x_1, \dots, x_k) = x$ . Since  $x \notin \mathcal{L}$ , it must be that  $x_i \notin \mathcal{L}$  for some  $i$ . Therefore, there exists an index  $i \in [k]$  such that  $\Pi$  is accepting and  $x_i \notin \mathcal{L}$  with probability at least  $\epsilon/k$ . In fact, this holds for a uniformly random  $i \in [k]$ .

We now consider an experiment in which the public key of the top-level BARG is programmed on  $i$ . Since a programmed key is computationally indistinguishable from an honestly generated key, and since  $x_i$  can be decided in time  $T(x_i) \leq T(x) \leq \text{poly}(\lambda)$ , it follows that in this modified experiment the event  $x_i \notin \mathcal{L}$  still occurs with probability at least  $\epsilon/k - \text{negl}(\lambda)$ . Using the trapdoor, we can then extract from  $\tilde{\Pi}$  a valid witness, namely an accepting proof  $\Pi_i$  for  $x_i$ , with probability at least  $\epsilon/k - \text{negl}(\lambda)$ .

If  $\Pi_i$  is a proof of the underlying SNARG we immediately obtain a contradiction to its soundness. Otherwise,  $\Pi_i$  is itself a merged proof, and we may continue the extraction process recursively. However, since the probability of extracting a proof for a false statement decreases by a factor of  $k$  at each level, we must restrict the merge tree to have constant depth.

**Step 1: Mergeable proofs with polynomially-bounded depth.** As a first step, we construct mergeable SNARGs that support merge trees of polynomial depth  $d = d(\lambda)$ , where both the public key and the proof size grow with  $d$ . Our starting point is the constant-depth construction described above, with a public key that contains  $d$  independent BARG public keys, one for each level of the merge tree. As discussed, the straightforward soundness analysis incurs a multiplicative loss of  $k$  at each extraction step, leading to a negligible success probability  $k^{-d}$ . We overcome this loss by adopting an extraction strategy that depends explicitly on the structure of the merge tree.

Suppose again that an adversary  $\text{Adv}$  produces a false statement  $x \notin \mathcal{L}$  with  $T(x), W(x) \leq \text{poly}(\lambda)$ , together with an accepting proof  $\Pi$ , with non-negligible probability  $\epsilon$ . Intuitively, assume that  $\text{Adv}$  constructs  $x$  by merging  $n$  statements  $y_1, \dots, y_n$  according to some merge tree of depth  $d$ . Since  $x \notin \mathcal{L}$ , at least one leaf must contain a false statement  $y_t \notin \mathcal{L}$ . The goal of the reduction is to extract an accepting SNARG proof for such a leaf.

Although a tree of depth  $d$  may have up to  $k^d$  leaves, the fact that the instance merger is tree-bounded ensures that  $n \leq W(x)$  is polynomially bounded. Therefore, if we extract along a path to a uniformly random leaf  $y_r$ , we obtain a false statement together with an accepting SNARG proof with probability at least  $\epsilon/n$ , which is still non-negligible.

This intuition can be formalized in the *static setting*, where both the merged instances and the structure of the merge tree are fixed independently of the public key. The adaptive setting, however, introduces additional technical challenges. We first address the issue of adaptive instances under the simplifying assumption that the merge tree structure is fixed, and then extend the argument to the fully adaptive setting in which the merge tree itself may depend on the public key.

**Challenge I: Dealing with Adaptive instances.** We begin by describing a solution under the simplifying assumption that  $\text{Adv}$  constructs  $x$  according to a merge tree with a fixed structure, regardless of the public key. The merged instances themselves, however, may still be chosen adaptively as a function of the public key.

In this setting, the soundness reduction selects a random leaf  $y_r$  of level  $h \leq d$  and programs the BARG public key at each level  $\ell \leq h$  on an index  $r_\ell \in [k]$  along the path to  $y_r$ . If  $\text{Adv}$  outputs an accepting proof  $\Pi$ , the reduction recursively extracts proofs  $\Pi_0, \dots, \Pi_h$  where  $\Pi = \Pi_0$ . Let  $x_\ell$  denote the instance certified by  $\Pi_\ell$ , so that  $x_0 = x$  and  $x_h = y_r$ .

By the somewhere-extraction property of the BARG, except with negligible probability each  $\Pi_\ell$  is accepting (in what follows, we assume that this is indeed the case). In particular,  $\Pi_h$  is a SNARG proof at a leaf of the merge tree, while  $\Pi_0, \dots, \Pi_{h-1}$  are merged proofs. For each  $\ell < h$ , let  $x_{\ell,1}, \dots, x_{\ell,k}$  denote the instances contained in  $\Pi_\ell$ . Since  $\Pi_\ell$  is accepting, we have that  $x_\ell = \text{IM}(x_{\ell,1}, \dots, x_{\ell,k})$ , and since the BARG public key at level  $\ell$  is programmed on  $r_\ell$  we have that  $x_{\ell+1} = x_{\ell, r_{\ell+1}}$ .

The reduction ultimately extracts an accepting SNARG proof for the leaf  $y_r$ . Thus, to break the soundness of the underlying SNARG, it suffices to argue that  $y_r \notin \mathcal{L}$  with noticeable probability. Intuitively, this should follow from the fact that the programmed path is hidden from  $\text{Adv}$ . Indeed, if  $\text{Adv}$  could identify the programmed leaf, it might ensure that  $y_r \in \mathcal{L}$  by placing all false statements on other branches.

The problem with this intuition is that the reduction observes only the root instance  $x$  and, apart from the extracted leaf  $y_r$ , it does not know the instances at the other leaves which are chosen adaptively by  $\text{Adv}$ . Consequently, even if  $\text{Adv}$  always ensures that  $y_r \in \mathcal{L}$ , the reduction cannot immediately violate key indistinguishability, since it does not know which leaves contain false statements. To address this, we employ a more refined, level-by-level analysis.

Let  $n_\ell$  denote the number of leaves in the subtree rooted at the  $\ell$ -th node along the extracted path. We refer to  $n_\ell$  as the *weight* of that node. Intuitively, since  $x \notin \mathcal{L}$ , there must exist at least one root-to-leaf path consisting entirely of false statements. Because the reduction extracts along a path to a random leaf  $y_r$ , we expect that at each level  $\ell$ , the probability that  $x_\ell \notin \mathcal{L}$  is at least inversely proportional to  $n_\ell$ .

To formalize this intuition, define a random variable  $\alpha_\ell$  as follows: if  $x_\ell \notin \mathcal{L}$ , then  $\alpha_\ell = 1/n_\ell$ ; otherwise,  $\alpha_\ell = 0$ . (If  $\Pi$  is rejecting, we also set  $\alpha_\ell = 0$ .) To argue soundness it is sufficient to prove that the expectation of each  $\alpha_\ell$  is at least  $\epsilon/n - \text{negl}(\lambda)$ . Since  $\alpha_\ell \leq 1$ , this implies that  $x_\ell \notin \mathcal{L}$  and  $\Pi_\ell$  is accepting with probability at least  $\epsilon/n - \text{negl}(\lambda)$ . In particular, for  $\ell = h$ ,  $\Pi_h$  is a SNARG proof at a leaf, yielding a contradiction to the soundness of the underlying SNARG.

We bound the expectation of  $\alpha_\ell$  inductively. For  $\ell = 0$ , we have  $x_0 = x \notin \mathcal{L}$  and  $\Pi_0$  is accepting with probability  $\epsilon$ , while the total number of leaves is  $n$ . Hence,  $\mathbb{E}[\alpha_0] = \epsilon/n$ . It remains to show that  $\mathbb{E}[\alpha_\ell - \alpha_{\ell+1}] \leq \text{negl}(\lambda)$  for every  $\ell < h$ . Since  $\alpha_{\ell+1} \geq 0$ , it is enough to bound this expectation conditioned on the event  $G$  that  $\alpha_\ell > 0$  (i.e. that  $x_\ell \notin \mathcal{L}$ ).

Fix the indices  $r_1, \dots, r_\ell$  programmed in the first  $\ell$  public keys. This determines the first  $\ell$  nodes along the extracted path and, in particular, fixes the weight  $n_\ell$  of the subtree rooted at  $x_\ell$  and the weights of its children  $n_{\ell,1}, \dots, n_{\ell,k}$ . Since the reduction extracts along a path to a random leaf, for every  $i \in [k]$  we have that  $r_{\ell+1} = i$  with probability  $n_{\ell,i}/n_\ell$ . Therefore:

$$\begin{aligned} \mathbb{E}[\alpha_\ell - \alpha_{\ell+1} \mid G] &\leq \frac{1}{n_\ell} - \sum_i \frac{1}{n_\ell} \cdot \Pr[r_{\ell+1} = i \wedge x_{\ell,i} \notin \mathcal{L} \mid G] + \text{negl}(\lambda) \\ &= \frac{1}{n_\ell} - \sum_i \frac{1}{n_\ell} \cdot \frac{n_{\ell,i}}{n_\ell} \cdot \Pr[x_{\ell,i} \notin \mathcal{L} \mid G \wedge r_{\ell+1} = i] + \text{negl}(\lambda) \\ &\leq 1 - \sum_i \Pr[x_{\ell,i} \notin \mathcal{L} \mid G \wedge r_{\ell+1} = i] + n_\ell \cdot \text{negl}(\lambda). \end{aligned} \tag{1}$$

Conditioned on  $G$ , we have  $x_\ell \notin \mathcal{L}$ . Intuitively, this implies that at least one of the children  $x_{\ell,i}$  must also lie outside  $\mathcal{L}$ , and therefore the sum in (1) should be at least 1. However, we must first argue that conditioning on the event  $r_{\ell+1} = i$  does not significantly decrease the probability that  $x_{\ell,i} \notin \mathcal{L}$ .

To this end, consider an experiment in which only the first  $\ell$  public keys are programmed, and all subsequent keys are generated honestly. In this experiment, we can still extract the proofs  $\Pi_1, \dots, \Pi_\ell$ . Since the first  $\ell$  keys are distributed exactly as in the original reduction, key indistinguishability implies that the probability of the event  $x_{\ell,i} \notin \mathcal{L}$  conditioned on  $G \wedge r_\ell = i$  changes only negligibly. Moreover, in this modified experiment, the adversary's view is independent of  $r_{\ell+1}$ . Hence,

$$\sum_i \Pr [x_{\ell,i} \notin \mathcal{L} \mid G \wedge r_{\ell+1} = i] = \sum_i \Pr [x_{\ell,i} \notin \mathcal{L} \mid G] \geq 1.$$

Therefore, this sum must be at least  $1 - \text{negl}(\lambda)$  in the original reduction as well. Substituting back into (1), we conclude that  $\mathbb{E}[\alpha_\ell - \alpha_{\ell+1}] \leq \text{negl}(\lambda)$ .

**Challenge II: Dealing with Adaptive merge tree structure.** The reduction described above programs a path to a random leaf of the merge tree into the public key. This strategy relies on the assumption that the structure of the merge tree is fixed in advance. In general, however, the adversary  $\text{Adv}$  may construct a merge tree whose structure depends on the public key.<sup>2</sup>

In this fully adaptive setting, it appears impossible to program the BARG public keys in advance so as to extract a false statement with non-negligible probability. To illustrate the difficulty, consider an adversary that constructs a merge tree with a “centipede” structure: a binary tree of height  $\lambda$  with exactly one leaf at each level, positioned either on the left or on the right. All leaves contain valid SNARG proofs, but only the leaf at the deepest level corresponds to a false statement. To extract this proof, the reduction would need to program the BARG public keys correctly at all  $\lambda$  levels along the unique path to the bottom-most leaf. If the adversary chooses one of the  $2^\lambda$  possible such trees at random, using randomness derived from the public key, then any fixed programming strategy succeeds only with negligible probability.

To overcome this barrier, we abandon the idea of programming a fixed path into the public key. Instead, we extract the path *dynamically* as the merge tree is formed.

**Functional BARGs.** Our solution relies on a stronger variant of rate-1 BARGs, known as *functional BARGs*, introduced and constructed in [BCJP24] under LWE. In a functional BARG, the public key is programmed on a function  $\mathcal{F}$  rather than a single index. Moreover, both the prover and verifier are given as an additional shared input a *tag*  $z$ , and completeness must hold for every choice of  $z$ .

The knowledge-extraction property of functional BARGs again consists of two components. First, a public key programmed on  $\mathcal{F}$  is computationally indistinguishable from an honestly generated key. Second, given a programmed key  $\beta_{\mathcal{F}}$ , if an efficient adversary outputs a tag  $z$ , instances  $x_1, \dots, x_k$ , and an accepting proof  $\Pi$  under  $\beta_{\mathcal{F}}$  and  $z$ , then using the trapdoor one can efficiently extract from  $\Pi$  a valid witness for  $x_i$ , where  $i = \mathcal{F}(z)$ , except with negligible probability.

Using functional BARGs, the high-level idea of our construction is as follows. Rather than programming a fixed extraction path during CRS generation, path is determined dynamically as the merge tree is constructed. Concretely, when merging proofs  $\Pi_1, \dots, \Pi_k$  into a new proof  $\Pi$ , we generate the BARG proof using a tag  $z$  that encodes the *weights*  $n_1, \dots, n_k$  of the input proofs, where the weight of a proof corresponds to the number of leaves in the merge tree that produced it.

---

<sup>2</sup>In fact,  $\text{Adv}$  may generate its proof in an arbitrary manner without explicitly following any merge tree or prescribed merging algorithm.

In the soundness reduction, we program the BARG public key so that, given a tag  $z = (n_1, \dots, n_k)$ , each index  $i$  is sampled with probability proportional to the weight  $n_i$ . The key observation is that if each internal node independently selects which child to extract according to the children's weights, the result is that the reduction extracts along a path from the root to a *uniformly random leaf*, regardless of merge tree structure.

Following this idea, we modify the construction as follows. Each proof carries a weight  $n$  equal to the number of leaves in the merge tree that produced it. A fresh proof under the underlying SNARG has weight 1, and when merging proofs, their weights are summed. In addition to the merged instances  $x_1, \dots, x_k$ , a merged proof also includes the corresponding weights  $n_1, \dots, n_k$ .

The BARG proof  $\tilde{\Pi}$  is generated using the tag  $z = (n_1, \dots, n_k)$ , and it additionally certifies that each  $x_i$  has an accepting proof of weight  $n_i$  (i.e., that the weights recorded in the merged proof are consistent with the weights of the underlying proofs). During verification of a merged proof for  $x$ , the verifier uses the tag  $z$  included in the proof to verify  $\tilde{\Pi}$ . It also checks that  $n = \sum n_i$  and that  $n \leq W(x)$ .

In the soundness reduction, we sample the public key as follows. Let  $\mathcal{F}_\rho$  be the function that, given a tag  $z = (n_1, \dots, n_k)$ , uses randomness  $\rho$  to sample an index  $i \in [k]$  with probability proportional to  $n_i$ , and outputs  $i$ . For each level  $\ell$ , we sample independent randomness  $\rho_\ell$  and program the BARG public key at level  $\ell$  on  $\mathcal{F}_{\rho_\ell}$ . The remainder of the analysis then proceeds via the same level-by-level argument outlined above.

**Step 2: Mergeable arguments with unbounded depth.** In the construction described above, the public key contains one BARG public key for each level of the merge tree. Consequently, it supports only merge trees of a priori bounded depth. Our next goal is to construct a mergeable SNARG that supports an unbounded (polynomial) number of recursive merges. In particular, for any merge tree of depth  $d$ , the proof size grows linearly with  $d$ , and the public key size remains independent of  $d$ .

A natural approach to reducing the size of the public key is to reuse the same BARG public key at every level of the merge tree. Adopting this approach, we begin by introducing additional properties required from the underlying BARG in order to preserve succinctness and soundness.

**BARGs with strong succinctness.** As noted above, to maintain succinctness under recursive composition beyond constant depth, we rely on rate-1 BARGs. Such BARGs were proposed and constructed in [DGKV22, PP22], albeit with different parameter trade-offs. In the construction of [PP22], for a witness of size  $m$ , the public key has size  $\text{poly}(\lambda)$  and the proof has size  $m + m/\lambda + \text{poly}(\lambda)$ . In contrast, in [DGKV22], the public key has size  $\text{poly}(\lambda, m)$  and the proof has size  $m + \text{poly}(\lambda)$ .

Neither of these constructions suffices when the composition depth is unbounded. In the BARG of [PP22], the proof length increases by a multiplicative factor of  $(1 + 1/\lambda)$  at each level of composition, leading to an overall proof size that grows exponentially in  $d/\lambda$ . In the construction of [DGKV22], the public key size depends on the witness length  $m$ , and therefore can only support witnesses whose size is bounded a priori. Since, in our setting, the witness to each proof is itself a proof from the previous level, and since the proof size grows linearly with the depth, the number of supported levels is bounded by the public key size.

We therefore require BARGs whose proof size incurs only an additive  $\text{poly}(\lambda)$  overhead and whose public key size is independent of  $m$ . A construction with these parameters was recently

proposed in [DMP26] under LWE.

**BARGs with strong somewhere extraction.** In the soundness reduction above, we rely on the somewhere-extraction property of the BARG to recursively extract from merged proofs. However, when all levels share the same public key, the standard notion of somewhere extraction is no longer sufficient.

To see why, suppose an adversary  $\text{Adv}$  outputs an accepting merged proof  $\Pi$  containing a BARG proof under a public key  $\beta$ . Using the trapdoor  $\tau$ , we can extract from  $\Pi$  a proof  $\Pi'$  corresponding to an instance at the previous level. If  $\Pi'$  is itself a merged proof, we would like to continue extracting from it. The difficulty is that  $\Pi'$  was generated using the trapdoor  $\tau$ , whereas the standard somewhere-extraction guarantee only applies to proofs generated by an adversary given access to the public key alone.

One possible remedy would be to rely on a stronger *statistical* notion of somewhere extraction that holds even against unbounded adversaries [HW14]. However, existing constructions of rate-1 BARGs do not appear to satisfy such a strong guarantee. Instead, we adopt an intermediate notion in which the adversary is computationally bounded but is given access to both the BARG public key  $\beta$  and the trapdoor  $\tau$ . We refer to this property as *strong somewhere extraction*. Notably, the BARG construction of [DMP26] satisfies this strengthened notion. By applying the transformation of [BCJP24] to this construction, we obtain a rate-1 functional BARG with strong succinctness and strong somewhere extraction.

**Towards soundness with a single public key.** Equipped with strong BARGs, we now turn to the main technical challenge: establishing soundness when all levels of the merge tree share a single BARG public key. We begin by explaining why the previous soundness argument breaks down in this setting.

In essence, although we strengthened the somewhere-extraction property to hold even when the adversary is given the trapdoor, the key-indistinguishability property inherently fails once the trapdoor is revealed. In the earlier reduction, we relied on key indistinguishability to argue that programming the public keys for the top levels, while leaving the remaining keys honestly generated, does not significantly alter the distribution of extracted proofs. This hybrid argument crucially depends on having independent public keys at different levels. When all levels use the same public key the previous argument does not go through.

**The two-key technique.** To restore the soundness proof, we employ a two-key strategy. At a high level, we use one BARG public key to perform extraction at a given level, while programming the other key to extract one of the proofs at the next level. We then alternate the roles of the two keys as we recurse down the merge tree, until we extract a SNARG proof at a leaf. This approach is reminiscent of the two-key technique of Kalai, Raz, and Rothblum [KRR13], which underlies all constructions of SNARGs from standard assumptions. Our implementation, however, differs significantly from prior work due to the distributed nature of our proofs.

We now describe the construction in more detail. The public key of the mergeable SNARG consists of the public key of the underlying SNARG together with two BARG public keys,  $\beta_1$  and  $\beta_2$ . Each proof  $\Pi$  in the new system is a pair  $(\Pi_1, \Pi_2)$  of proofs for the same statement, where each  $\Pi_j$  is constructed as in the single-key construction described above, using the same BARG public key  $\beta_j$  at every level of the merge tree.

More concretely, the proof merger is given instances  $x_1, \dots, x_k$  and corresponding proofs  $\Pi_1, \dots, \Pi_k$ , where each  $\Pi_i = (\Pi_{i,1}, \Pi_{i,2})$ . It outputs a merged proof  $\Pi = (\Pi_1, \Pi_2)$ , where for each  $\Pi_j$  is obtained by merging  $\Pi_{1,j}, \dots, \Pi_{k,j}$  according to the construction above, using the public key  $\beta_j$  to generate the BARG proof. To verify a proof  $\Pi = (\Pi_1, \Pi_2)$ , the verifier checks each  $\Pi_j$  with respect to the underlying SNARG public key and the corresponding BARG public key  $\beta_j$ .

To make this construction sound, we must additionally ensure consistency between  $\Pi_1$  and  $\Pi_2$ . As described so far, the verifier checks each component separately, and therefore the two proofs might certify the same top-level statement, but following different merge trees. While the verifier can explicitly check consistency at the top level, namely, that both  $\Pi_1$  and  $\Pi_2$  contain the same instances  $x_1, \dots, x_k$  and weights  $n_1, \dots, n_k$ , it cannot verify consistency at deeper levels of the merge tree.

To address this issue, we augment each proof with a collision-resistant hash of the entire merge tree. This technique is analogous to the hash-and-BARG approach of [CJJ21, KVZ21], implementing the two-key paradigm using BARGs.

We incorporate a digest into each proof, in a manner analogous to how we incorporated weights, except that we now form a hash tree rather than an addition tree. For a proof  $\Pi = (\Pi_1, \Pi_2)$ , the hash tree is computed separately for  $\Pi_1$  and  $\Pi_2$ . Specifically, a fresh proof under the underlying SNARG includes the digest value  $d = \perp$ . When merging proofs for instances  $x_1, \dots, x_k$  with weights  $n_1, \dots, n_k$  and digests  $d_1, \dots, d_k$ , the merged proof contains the tuples  $(x_i, n_i, d_i)$  for all  $i \in [k]$ . The digest  $d$  of the merged proof is computed by hashing the values  $(x_i, n_i, d_i)_{i \in [k]}$ .

We also modify the BARG proof  $\tilde{\Pi}$  so that it additionally certifies that each instance  $x_i$  has an accepting proof with weight  $n_i$  and digest  $d_i$ . In other words, the weights and digests recorded in the merged proof match those of the underlying proofs. Finally, when verifying a merged proof  $\Pi = (\Pi_1, \Pi_2)$ , the verifier checks that both components contain the same digest  $d$ , and that this digest is correctly computed as the hash of the values  $(x_i, n_i, d_i)_{i \in [k]}$  appearing in each  $\Pi_j$ .

Intuitively, these checks ensure that if both BARG public keys are programmed on the same function and  $\Pi$  is accepting, then the two extracted paths, one from  $\Pi_1$  and one from  $\Pi_2$ , include the same sequence of instances and weights at every level.

**The soundness reduction.** Suppose again that an adversary  $\text{Adv}$  produces a false statement  $x \notin \mathcal{L}$  with  $\text{T}(x), \text{W}(x) \leq Q(\lambda)$  for some polynomial  $Q(\lambda)$ , together with an accepting proof  $\Pi = (\Pi_1, \Pi_2)$ , with non-negligible probability  $\epsilon$ .

Recall that in the bounded-depth construction described above, the reduction programs the BARG public key at each level  $\ell$  with a function  $\mathcal{F}_{\rho_\ell}$ , where the randomness  $\rho_\ell$  is chosen independently at each level. In the present construction, however, we program only one of the BARG public keys, say  $\beta_1$ , with a single function  $\mathcal{F}$  that emulates the behavior of the functions  $\mathcal{F}_{\rho_1}, \dots, \mathcal{F}_{\rho_d}$ .

A natural idea would be to derive the randomness  $\rho_\ell$  by evaluating a PRF on the level index  $\ell$ . However, since the merge tree is constructed bottom-up, the distance of a merged proof from the root is not known at the time it is created. Instead, we derive the randomness from the tag  $z = (n_1, \dots, n_k)$ , which encodes the weights of the merged proofs. More precisely, the programmed function  $\mathcal{F}[f_s]$  contains a hard-coded PRF  $f_s : \mathbb{N} \rightarrow \{0, 1\}^*$  with secret key  $s$ . Given a tag  $z = (n_1, \dots, n_k)$ , it computes the total weight  $n = \sum_i n_i$ , sets  $\rho = f_s(n)$ , and outputs  $\mathcal{F}_\rho(z)$ .

If  $\text{Adv}$  outputs an accepting proof  $\Pi = (\Pi_1, \Pi_2)$ , the reduction recursively extracts a sequence of proofs  $\Pi_{1,0}, \dots, \Pi_{1,h}$  from  $\Pi_1$ , where  $\Pi_{1,0} = \Pi_1$ . For each extracted proof  $\Pi_{1,\ell}$ , let  $x_\ell$  denote the certified instance and let  $n_\ell$  denote the weight of the proof. By the somewhere-extraction property

of the BARG, except with negligible probability each  $\Pi_{1,\ell}$  is accepting (in what follows, we assume that this is indeed the case). In particular, we obtain a strictly decreasing sequence of weights

$$Q(\lambda) \geq W(x) \geq n_0 > \dots > n_h = 1 \ .$$

That is,  $\Pi_{1,h}$  is a SNARG proof at a leaf of the merge tree, while  $\Pi_{1,0}, \dots, \Pi_{1,h-1}$  are merged proofs. For each  $\ell < h$ , we denote by  $x_{\ell,1}, \dots, x_{\ell,k}$  and  $n_{\ell,1}, \dots, n_{\ell,k}$  the instances and weights appearing in  $\Pi_{1,\ell}$ , satisfying

$$x_\ell = \text{IM}(x_{\ell,1}, \dots, x_{\ell,k}) \quad , \quad n_\ell = \sum_i n_{\ell,i} \ .$$

Looking ahead, the strict decrease  $n_0 > n_1 > \dots > n_h$  ensures that the randomness used at each level is derived from distinct inputs to the PRF. Moreover, this structure allows us to “deprogram” the public key and erase all the randomness below some level  $\ell$  by puncturing the PRF key below some weight  $n_\ell$ .

**Main claim.** Our analysis relies on the following claim: Fix a weight  $1 \leq n \leq Q(\lambda)$  and consider two functions  $f$  and  $f'$  that agree on all inputs strictly larger than  $n$ . We compare two experiments that follow the soundness reduction, except that in one experiment the BARG public key  $\beta_1$  is programmed on  $\mathcal{F}[f]$ , while in the other it is programmed on  $\mathcal{F}[f']$ . Recall that given a tag  $z = (n_1, \dots, n_k)$ , the function  $\mathcal{F}[g]$  outputs  $\mathcal{F}_\rho(z)$  where  $\rho = g(n)$  and  $n = \sum_i n_i$ . In each experiment, if the proof  $\Pi$  is rejecting, the output is  $\perp$ . Otherwise, let  $\ell(n) \leq h$  be the smallest index such that  $n_{\ell(n)} \leq n$ . That is,  $\ell(n)$  is the level of the first extracted proof with weight is at most  $n$ . The experiment outputs the instances and weights corresponding to the first  $\ell(n)$  nodes along the extracted path, together with the instances and weights of their siblings in the merge tree:

$$\{x_\ell, n_\ell\}_{\ell \leq \ell(n)} \quad , \quad \{x_{\ell,i}, n_{\ell,i}\}_{\ell \leq \ell(n), i \leq k} \ .$$

We claim that the outputs of these two experiments are computationally indistinguishable. Intuitively, this holds because all proofs extracted up to level  $\ell(n)$  originate from nodes whose weights exceed  $n$ . In particular, the proof at level  $\ell(n)$  was extracted from a node at level  $\ell(n) - 1$  whose weights is  $n_{\ell(n)-1} > n$ . Therefore the functions  $f$  and  $f'$  behave identically on the relevant inputs.

To prove the claim, we consider the following sequence of hybrid experiments:

**Hybrid 1:** This is the first experiment, in which  $\beta_1$  is programmed on  $\mathcal{F}[f]$ , while  $\beta_2$  is generated honestly.

**Hybrid 2:** In this hybrid, we additionally program  $\beta_2$  on  $\mathcal{F}[f']$ . Since we use the trapdoor of  $\beta_1$  to recursively extract from  $\Pi_1$ , the output of the experiment is computed without using the trapdoor of  $\beta_2$ . Therefore, the key-indistinguishability property of the BARG implies that the output distribution remains indistinguishable.

**Hybrid 3:** In this hybrid the output is computed by recursively extracting from  $\Pi_2$  rather than from  $\Pi_1$  using the trapdoor of  $\beta_2$ . Since  $\Pi_1$  and  $\Pi_2$  contain the same digest, and since  $f$  and  $f'$  agree on all inputs greater than  $n$ , the collision resistance of the hash function together with the somewhere-extraction property imply that the output is identical to the output of the previous experiment with all but negligible probability.

**Hybrid 4:** In this hybrid, we program both  $\beta_1$  and  $\beta_2$  on  $\mathcal{F}[f']$ . Since the output of the experiment is computed without using the trapdoor of  $\beta_1$ , the key-indistinguishability property of the BARG implies that the output distribution remains indistinguishable.

**Hybrid 5:** In this hybrid, we revert to extracting from  $\Pi_1$  (using the trapdoor of  $\beta_1$ ) instead of from  $\Pi_2$ . Since  $\Pi_1$  and  $\Pi_2$  contain the same digest, and both public keys are programmed on the same function  $\mathcal{F}[f']$ , collision resistance and the somewhere-extraction property again imply that the output is identical to the output of the previous experiment with all but negligible probability.

**Hybrid 6:** This final hybrid is the second experiment, in which  $\beta_1$  is programmed on  $\mathcal{F}[f']$  and  $\beta_2$  is generated honestly. Since the output of the experiment is computed without using the trapdoor of  $\beta_2$ , key indistinguishability implies that the output distribution remains indistinguishable.

Using the main claim established above, we now complete the soundness analysis. Recall that Adv outputs a proof  $\Pi = (\Pi_1, \Pi_2)$ , and the reduction recursively extracts a sequence of proofs  $\Pi_{1,0}, \dots, \Pi_{1,h}$  from  $\Pi_1$ . From this point onward, we no longer rely on  $\Pi_2$  (its role was solely to prove the main claim). For notational simplicity, we therefore drop the index  $j \in \{1, 2\}$  and write  $\Pi$  in place of  $\Pi_1$ , and  $\Pi_\ell$  in place of  $\Pi_{1,\ell}$ .

Recall that if  $\Pi$  is accepting, then we obtain a strictly decreasing sequence of weights  $Q(\lambda) \geq n_0 > \dots > n_h = 1$  and for every  $1 \leq n \leq Q(\lambda)$ , we define  $\ell(n)$  to be the smallest index  $0 \leq \ell(n) \leq h$  such that  $n_{\ell(n)} \leq n$ . We now associate a random variable  $\alpha_n$  with each weight  $n$ . Specifically, if  $\Pi$  is accepting and  $x_{\ell(n)} \notin \mathcal{L}$ , we set  $\alpha_n = 1/n_{\ell(n)}$ , and otherwise we set  $\alpha_n = 0$ .

For  $n = 1$ , we have  $n_h = n < n_{h-1}$ , and therefore  $\ell(1) = h$ . Consequently,  $\Pi_{\ell(1)} = \Pi_h$  is a SNARG proof at a leaf of the merge tree. Thus, if we can show that  $\mathbb{E}[\alpha_1]$  is non-negligible, it follows that  $\Pi_h$  is an accepting SNARG proof for a false statement with non-negligible probability, contradicting the soundness of the underlying SNARG.

We claim that  $\mathbb{E}[\alpha_n] \geq \epsilon/Q(\lambda) - \text{negl}(\lambda)$  for every  $1 \leq n \leq Q(\lambda)$ . For  $n = Q(\lambda)$  we have that  $n_0 \leq n$  and, therefore,  $\ell(Q(\lambda)) = 0$ . Since  $x_0 \notin \mathcal{L}$  and  $\Pi_0$  is accepting with probability  $\epsilon$  we have that  $\mathbb{E}[\alpha_{Q(\lambda)}] = \epsilon/Q(\lambda)$ . Therefore, it is sufficient to prove that for every  $1 < n \leq Q(\lambda)$ :

$$\mathbb{E}[\alpha_n - \alpha_{n-1}] \leq \text{negl}(\lambda) .$$

Recall that  $\ell(n)$  is the smallest index such that  $n_{\ell(n)} \leq n$ . Therefore, for any  $1 < n \leq Q(\lambda)$ , exactly one of the following holds:

- Either  $n_{\ell(n)} \leq n - 1$ , in which case  $\ell(n - 1) = \ell(n)$  and hence  $\alpha_n = \alpha_{n-1}$ ,
- Or  $n_{\ell(n)} = n$ , in which case  $\ell(n - 1) = \ell(n) + 1$ .

Let  $G$  denote the event that  $\alpha_n > 0$  and  $n_{\ell(n)} = n$ . If  $\alpha_n > \alpha_{n-1}$ , then necessarily  $G$  occurs. Therefore, it suffices to prove that

$$\mathbb{E}[\alpha_n - \alpha_{n-1} \mid G] \leq \text{negl}(\lambda) .$$

Let  $\ell = \ell(n)$ . Conditioned on the event  $G$ , we have  $n_\ell = n$ ,  $\ell(n - 1) = \ell + 1$ , and  $x_\ell \notin \mathcal{L}$  (since  $\alpha_n > 0$ ). Recall that the reduction programs the BARG public key on the function  $\mathcal{F}[f_s]$ , where  $f_s$

is a PRF with randomly chosen key  $s$ . In particular, the proof  $\Pi_{\ell+1}$  extracted from  $\Pi_\ell$  corresponds to the instance  $x_{\ell+1} = x_{\ell,r}$  for  $r = \mathcal{F}[f_s](z) = \mathcal{F}_\rho(z)$  where  $z = (n_{\ell,1}, \dots, n_{\ell,k})$  and  $\rho = f_s(n)$ .

Intuitively, to bound the conditional expectation above, we must argue that  $\rho = f_s(n)$  is computationally indistinguishable from uniform and independent of the instances and weights appearing in  $\Pi_\ell$ . At first glance, this should follow from the pseudorandomness of the PRF. However, there is a subtlety: the BARG public key is programmed on a function that hard-codes the PRF key  $s$ , and the reduction uses the corresponding trapdoor to extract  $\Pi_\ell$ .

To address this issue, we analyze a sequence of hybrid experiments. Consider an experiment  $\text{Ex}_1$  in which, instead of programming the public key on  $\mathcal{F}[f_s]$ , we program it on  $\mathcal{F}[f_{s[n],\rho}]$ , where  $f_{s[n],\rho}$  is defined as follows: on input  $n$ , it outputs  $\rho = f_s(n)$ , and on any other input it evaluates the PRF using the punctured key  $s[n]$ . Since  $f_s$  and  $f_{s[n],\rho}$  agree on all inputs strictly larger than  $n$ , it follows from the main claim that the expectation  $\mathbb{E}[\alpha_n - \alpha_{n-1} \mid G]$  in  $\text{Ex}_1$  is negligibly close to the expectation in the original security reduction. Thus, it suffices to show that

$$\mathbb{E}_{\text{Ex}_1}[\alpha_n - \alpha_{n-1} \mid G] \leq \text{negl}(\lambda) .$$

By the puncturable PRF security, the output  $f_s(n)$  remains pseudorandom even given the punctured key  $s[n]$ . Therefore, we may consider a second experiment  $\text{Ex}_2$  in which the public key is programmed on  $\mathcal{F}[f_{s[n],\rho}]$ , where  $\rho$  is chosen uniformly at random and independently of  $s$ . It thus suffices to prove that

$$\mathbb{E}_{\text{Ex}_2}[\alpha_n - \alpha_{n-1} \mid G] \leq \text{negl}(\lambda) .$$

In the experiment  $\text{Ex}_2$ , define a random variable  $\alpha'_{n-1}$  analogously to  $\alpha_{n-1}$ , except that instead of using the instance  $x_{\ell+1}$  and weight  $n_{\ell+1}$  associated with  $\Pi_{\ell+1}$ , we use the instance  $x_{\ell,r}$  and weight  $n_{\ell,r}$  appearing in  $\Pi_\ell$ , where  $r = \mathcal{F}_\rho(z)$ . That is,  $\alpha'_{n-1}$  is set to  $1/n_{\ell,r}$  if  $x_{\ell,r} \notin \mathcal{L}$ , and to 0 otherwise. Since the BARG public key is programmed on  $\mathcal{F}[f_{s[n],\rho}]$ , the somewhere-extraction property implies that  $\alpha'_{n-1} = \alpha_{n-1}$ . Therefore, it suffices to prove that

$$\mathbb{E}_{\text{Ex}_2}[\alpha_n - \alpha'_{n-1} \mid G] \leq \text{negl}(\lambda) .$$

In the final experiment  $\text{Ex}_3$ , we program the public key on  $\mathcal{F}[f_{s[n],\perp}]$ , where  $f_{s[n],\perp}$  is defined identically to  $f_{s[n],\rho}$  except that on input  $n$  it outputs  $\perp$  rather than  $\rho$ . Observe that both  $\alpha_n$  and  $\alpha'_{n-1}$  depend only on the values contained in  $\Pi_\ell$  and not on the extracted proof  $\Pi_{\ell+1}$ . Since  $f_{s[n],\perp}$  and  $f_{s[n],\rho}$  agree on all inputs strictly larger than  $n$ , the main claim implies that the expectation above changes only negligibly when moving from  $\text{Ex}_2$  to  $\text{Ex}_3$ . Thus, it suffices to show that

$$\mathbb{E}_{\text{Ex}_3}[\alpha_n - \alpha'_{n-1} \mid G] \leq \text{negl}(\lambda) .$$

In the experiment  $\text{Ex}_3$ , the adversary's view is completely independent of  $\rho$ . Fix all the randomness of the experiment except for  $\rho$  such that  $G$  occurs. Recall that  $r = \mathcal{F}_\rho(z)$ , where  $z = (n_{\ell,1}, \dots, n_{\ell,k})$ . By the definition of  $\mathcal{F}$ , for each  $i \in [k]$ , we have that  $r = i$  with probability

$n_{\ell,i}/n$ . Since  $G$  implies  $x_\ell \notin \mathcal{L}$ , there must exist some  $i \in [k]$  such that  $x_{\ell,i} \notin \mathcal{L}$ . It follows that

$$\begin{aligned} \mathbb{E}_\rho[\alpha_n - \alpha'_{n-1}] &= \frac{1}{n} - \sum_i \frac{1}{n_{\ell,i}} \cdot \Pr[r_\ell = i \wedge x_{\ell,i} \notin \mathcal{L}] \\ &= \frac{1}{n} - \sum_i \frac{1}{n_{\ell,i}} \cdot \Pr[r_\ell = i] \cdot \Pr[x_{\ell,i} \notin \mathcal{L}] \\ &= \frac{1}{n} - \sum_i \frac{1}{n_{\ell,i}} \cdot \frac{n_{\ell,i}}{n} \cdot \Pr[x_{\ell,i} \notin \mathcal{L}] \\ &\leq 1 - \sum_i \Pr[x_{\ell,i} \notin \mathcal{L}] \leq 0 . \end{aligned}$$

**Shrinking merged proofs.** We now explain how to shrink the merged proofs under the assumption that the constituent instances can be reconstructed from the merged instance. Let  $R$  be a polynomial-time reconstruction algorithm. Consider a merged proof  $\Pi$  that contains instances  $x_1, \dots, x_k$ , and let  $x = \text{IM}(x_1, \dots, x_k)$ . Suppose there exists advice  $z \in \{0, 1\}^*$  such that  $R(x, z) = (x_1, \dots, x_k)$ . We observe that the instances  $x_1, \dots, x_k$  embedded in  $\Pi$  can be replaced by the advice string  $z$ , without affecting verifiability. Given the merged instance  $x$ , the verifier can use  $z$  to reconstruct  $x_1, \dots, x_k$  and proceed with verification exactly as before.

## 2 Preliminaries

We use the following cryptographic primitives.

**Collision-resistant hashing.** A collision-resistant hash family is given by an algorithm  $\mathcal{H}$  that given as inputs the security parameter  $\lambda$  runs in time  $\text{poly}(\lambda)$  and samples a description of a polynomial-time hash function  $h : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ . We require that for every  $\text{poly}(\lambda)$ -size adversary  $\text{Adv}$  there exists a negligible function  $\mu$  such that for every  $\lambda \in k$ :

$$\Pr \left[ \begin{array}{l} h \leftarrow \mathcal{H}(\lambda) \\ (x_1, x_2) \leftarrow \text{Adv}(h) \end{array} : \begin{array}{l} x_1 \neq x_2 \\ h(x_1) = h(x_2) \end{array} \right] \leq \mu(\lambda) .$$

The existence of a collision-resistant hash family is implied by BARGs and one-way functions [BKP<sup>+</sup>23].

**Puncturable pseudorandom functions.** We give a simple notion of puncturable pseudorandom functions that is tailored to our applications. Such puncturable pseudorandom functions can be constructed from any one-way function [GGM84, BW13, BGI14, KPTZ13]. A puncturable pseudorandom functions is given by a family of functions  $\{f_s : [2^\lambda] \rightarrow \{0, 1\}^{\ell(\lambda)} : \lambda \in \mathbb{N}, s \in \{0, 1\}^\lambda\}$  computable in time  $\text{poly}(\lambda)$  where for every key  $s \in \{0, 1\}^\lambda$  and  $n \in [2^\lambda]$  we can compute in time  $\text{poly}(\lambda)$  a punctured key  $s[n]$  such that:

**Correctness.** For every  $n' > n$ ,  $f_{s[n]}(n') = f_s(n')$ .

**Pseudorandomness.** For every  $n \in [2^\lambda]$ ,  $(s[n], f_s(n)) \approx_c (s[n], \rho)$  where  $s \leftarrow \{0, 1\}^\lambda$  and  $\rho \leftarrow \{0, 1\}^{\ell(\lambda)}$ .

### 3 Batch Arguments

In this section we define the notion of batch arguments (BARGs) that we rely on in subsequent sections. The notion we define is based on the notion of somewhere-extractable BARGs for the index language [CJJ21], rate-1 BARGs [DGKV22, PP22], and BARGs with functional extraction [BCJP24]. Additionally we introduce the notion of strong somewhere extraction.

For every  $t \in \mathbb{N}$ , let  $\mathcal{L}_{\mathcal{U}}^t$  be the language that contains an instance  $(\mathcal{M}, z)$  if and only if the machine  $\mathcal{M}(z)$  accepts in at most  $t$  steps. Let  $\mathcal{R}_{\mathcal{B}}^t$  be the relation that contains an instance  $x = (\mathcal{M}, k)$  with witness  $w = (w_1, \dots, w_k)$  if and only if for every  $i \in [k]$ ,  $(\mathcal{M}, (i, w_i)) \in \mathcal{L}_{\mathcal{U}}^t$ .

A functional batch argument (BARG) consist of algorithms  $(\mathsf{G}, \mathsf{P}, \mathsf{V}, \mathsf{E})$  with the following syntax:

- G:** The randomized generation algorithm takes as input the security parameter  $\lambda$  and a machine  $\mathcal{F}$ . It outputs a public key  $\beta$  and a secret key  $\tau$ .
- P:** The deterministic prover algorithm takes as input the public key  $\beta$ , a tag  $z$ , instance  $x$  and a witness  $w$ . It outputs a proof  $\Pi$ .
- V:** The deterministic verifier algorithm takes as input the public key  $\beta$ , a tag  $z$ , instance  $x$ , and a proof  $\Pi$ . It outputs an acceptance bit.
- E:** The deterministic extraction algorithm takes as input the secret key  $\tau$  and a proof  $\Pi$ . It outputs a witness  $w \in \{0, 1\}^*$ .

**Definition 3.1.** *A functional BARG  $(\mathsf{G}, \mathsf{P}, \mathsf{V}, \mathsf{E})$  satisfies the following requirements:*

**Completeness.** *For every  $\lambda \in \mathbb{N}$ , machine  $\mathcal{F}$ , tag  $z \in \{0, 1\}^*$  and  $(x = (\mathcal{M}, k), w \in \{0, 1\}^{m \times k}) \in \mathcal{R}_{\mathcal{B}}^t$  such that  $|\mathcal{F}|, |z|, |\mathcal{M}|, k, t \leq 2^\lambda$  we have that:*

$$\Pr \left[ \begin{array}{l} (\beta, \tau) \leftarrow \mathsf{G}(\lambda, \mathcal{F}) \\ \Pi \leftarrow \mathsf{P}(\beta, z, x, w) \end{array} : \mathsf{V}(\beta, z, x, \Pi) = 1 \right] = 1 .$$

**Key Indistinguishability.** *For every  $\text{poly}(\lambda)$ -size adversary  $\text{Adv}$  and polynomial  $Q(\lambda)$  there exists a negligible function  $\mu$  such that for every  $\lambda \in \mathbb{N}$  and a pair of machines  $\mathcal{F}_1, \mathcal{F}_2$  such that  $|\mathcal{F}_1| = |\mathcal{F}_2| \leq Q(\lambda)$ :*

$$\Pr \left[ \begin{array}{l} b \leftarrow \{0, 1\} \\ (\beta, \tau) \leftarrow \mathsf{G}(\lambda, \mathcal{F}_b) \end{array} : \text{Adv}(\beta) = b \right] \leq \frac{1}{2} + \mu(\lambda) .$$

**Somewhere Extraction.** *For every  $\text{poly}(\lambda)$ -size adversary  $\text{Adv}$  and polynomial  $Q(\lambda)$  there exists a negligible function  $\mu$  such that for every  $\lambda \in \mathbb{N}$  and  $t, k \in [Q(\lambda)]$ :*

$$\Pr \left[ \begin{array}{l} (\beta, \tau) \leftarrow \mathsf{G}(\lambda, \mathcal{F}) \\ (z, \mathcal{M}, \Pi) \leftarrow \text{Adv}(\beta) \\ w \leftarrow \mathsf{E}(\tau, \Pi) \end{array} : \begin{array}{l} \mathsf{V}(\beta, z, (\mathcal{M}, k), \Pi) = 1 \\ (\mathcal{M}, (\mathcal{F}(z), w)) \notin \mathcal{L}_{\mathcal{U}}^t \end{array} \right] \leq \mu(\lambda) .$$

*We say that the BARG satisfies strong somewhere extraction if the above holds even when  $\text{Adv}$  is given both keys  $\beta$  and  $\tau$ .*

**Efficiency.** *In the completeness experiment above:*

- The running time of  $\mathsf{G}$  in the completeness experiment is  $\text{poly}(\lambda, |\mathcal{F}|)$ .
- The running time of  $\mathsf{P}$  in the completeness experiment is  $\text{poly}(\lambda, |\mathcal{F}|, |z|, |\mathcal{M}|, k, t, m)$  and  $|\Pi| = \text{poly}(\lambda, m)$ . We say that the BARG has rate 1 if  $|\Pi| = m + \text{poly}(\lambda)$ .
- The running time of  $\mathsf{V}$  in the somewhere-extraction experiment is  $|(z, \mathcal{M}, \Pi)| \cdot \text{poly}(\lambda)$ .
- The running time of  $\mathsf{E}$  in the somewhere-extraction experiment is  $\text{poly}(\lambda, |\Pi|)$ .

A plain (as opposed to functional) BARG satisfies the definition of a functional BARG when  $\mathcal{F}$  is restricted to be a constant function.

**Theorem 3.2.** *Assuming the hardness of the learning with errors problem, there exist rate-1 functional BARGs with strong somewhere extraction.*

*Proof sketch.* Rate-1 BARGs (with public key and proof length given by Definition 3.1) were recently constructed in [DMP26]. These BARGs also satisfy with strong somewhere extraction. The work of [BCJP24] suggested a transformation from (plain) BARGs to functional ones, preserving rate and strong extraction. Therefore, the required BARGs follow by applying the transformation of [BCJP24] to the BARGs of [DMP26].  $\square$

## 4 Mergeable Arguments

In this section we define the notion of a mergeable SNARG and state our main result (Theorem 4.6). Let  $\mathcal{L}$  be a language decidable by a deterministic algorithm  $A$  and let  $\mathsf{T} : \{0, 1\}^* \rightarrow \mathbb{N}$  be an efficiently computable function such that for every  $x \in \{0, 1\}^*$ ,  $\mathsf{T}(x) \geq |x|$  and  $A(x)$  runs in time at most  $\mathsf{T}(x)$ .

A succinct non-interactive argument (SNARG) for  $\mathcal{L}$  consist of algorithms  $(\mathsf{G}, \mathsf{P}, \mathsf{V})$  with the following syntax:

**G:** The randomized generation algorithm takes as input the security parameter  $\lambda$ . It outputs a public key  $\sigma$ .

**P:** The deterministic prover algorithm takes as input the public key  $\sigma$  and  $x \in \mathcal{L}$ . It outputs a proof  $\Pi$ .

**V:** The deterministic verifier algorithm takes as input the public key  $\sigma$ ,  $x$  and a proof  $\Pi$ . It outputs a bit.

**Definition 4.1** (SNARG for  $\mathcal{L}$ ). *A non-interactive argument  $(\mathsf{G}, \mathsf{P}, \mathsf{V})$  for  $\mathcal{L}$  satisfies the following requirements:*

**Completeness:** *For every  $\lambda \in \mathbb{N}$  and  $x \in \mathcal{L}$  such that  $\mathsf{T}(x) \leq 2^\lambda$ :*

$$\Pr \left[ \begin{array}{l} \sigma \leftarrow \mathsf{G}(\lambda) \\ \Pi \leftarrow \mathsf{P}(\sigma, x) \end{array} : \mathsf{V}(\sigma, x, \Pi) = 1 \right] = 1 .$$

**Computational Soundness:** For every  $\text{poly}(\lambda)$ -size adversary  $\text{Adv}$  and polynomial  $\bar{T}$  there exists a negligible function  $\mu$  such that for every  $\lambda \in \mathbb{N}$ :

$$\Pr \left[ \begin{array}{l} \sigma \leftarrow G(\lambda) \\ (x, \Pi) \leftarrow \text{Adv}(\sigma) \end{array} : \begin{array}{l} x \notin \mathcal{L} \\ T(x) \leq \bar{T}(\lambda) \\ V(\sigma, x, \Pi) = 1 \end{array} \right] < \mu(\lambda) .$$

**Efficiency:**

- The running time of  $G$  in the completeness experiment is  $\text{poly}(\lambda)$ .
- The running time of  $P$  in the completeness experiment is  $\text{poly}(\lambda, T(x))$  and  $|\Pi| = \text{poly}(\lambda)$ .
- The running time of  $V$  in the soundness experiment is  $\text{poly}(\lambda, |x|, |\Pi|)$ . We say that the SNARK has a linear verifier if the running time of  $V$  is  $(|x| + |\Pi|) \cdot \text{poly}(\lambda)$ .

**Theorem 4.2** ([CJJ21, BKP<sup>+</sup>23]). Assuming one-way functions and BARGs, there exists a SNARG for  $\mathcal{L}$ .

*Proof sketch.* SNARG for  $\mathcal{L}$  are constructed in [CJJ21] based on BARGs and the notion of somewhere-extractable hash that can be instantiated by combining hash trees with BARGs. A collision-resistant hash family follows from BARGs and one-way functions [IKO05, BKP<sup>+</sup>23].  $\square$

**Definition 4.3** (Instance Merging). An polynomial-time algorithm  $\text{IM}$  is an instance merger for  $\mathcal{L}$  if for every  $x_1, \dots, x_k \in \mathcal{L}$ , it holds that for  $x = \text{IM}(x_1, \dots, x_k)$ :

- $x \in \mathcal{L} \cup \{\perp\}$ .
- If  $x \neq \perp$  then  $T(x_i) \leq T(x)$  for every  $i \in [k]$ .

**Definition 4.4** (Tree-Bounded Instance Merging). An instance merger  $\text{IM}$  is tree-bounded if there exists a polynomial-time computable weight function  $W : \{0, 1\}^* \rightarrow \mathbb{N}^+$  such that the following hold:

- For every  $x_1, \dots, x_k \in \{0, 1\}^*$  such that  $\text{IM}(x_1, \dots, x_k) \neq \perp$ ,  $W(\text{IM}(x_1, \dots, x_k)) \geq \sum W(x_i)$ .
- There exists a polynomial  $Q$  such that for every  $x \in \{0, 1\}^*$ ,  $W(x) \leq Q(T(x))$ .

Let  $\text{IM}$  be a tree-bounded instance merger for  $\mathcal{L}$ . A SNARG  $(G, P, V)$  is mergeable with respect to  $\text{IM}$  if there exists a deterministic proof merger algorithm  $\text{PM}$  that takes as input a public key  $\sigma$ , and a sequence of pairs  $(x_1, \Pi_1), \dots, (x_k, \Pi_k)$  and outputs a proof  $\Pi$ .

**Definition 4.5** (Proof Merger). A proof merger  $\text{PM}$  for  $(G, P, V)$  satisfies the following requirements:

**Merge completeness:** For every  $\lambda \in \mathbb{N}$ ,  $\sigma \in G(\lambda)$  and  $(x_1, \Pi_1), \dots, (x_k, \Pi_k)$  such that  $V(\sigma, x_\kappa, \Pi_\kappa) = 1$  for every  $\kappa \in [k]$ , let:

$$\begin{aligned} x &\leftarrow \text{IM}(x_1, \dots, x_k) , \\ \Pi &\leftarrow \text{PM}(\sigma, (x_1, \Pi_1), \dots, (x_k, \Pi_k)) . \end{aligned}$$

If  $x \neq \perp$  then  $V(\sigma, x, \Pi) = 1$ .

**Merge efficiency:** PM runs in polynomial time and in the completeness experiment above:

$$|\Pi| = \max |\Pi_i| + |(x_1, \dots, x_k)| \cdot \text{poly}(\lambda) .$$

**Theorem 4.6.** *Let IM be a tree-bounded instance merger for  $\mathcal{L}$ . Assuming rate-1 functional BARGs with strong somewhere extraction, there exists a mergeable SNARG for  $\mathcal{L}$  with respect to IM with a linear verifier.*

The construction of the proof system is described in Section 5 and Section 6 gives the analysis.

## 5 Construction

In this section we describe the construction of our mergeable SNARG for  $\mathcal{L}$ . See Section 1.2 for an overview.

Let IM be a tree-bounded instance merger for  $\mathcal{L}$  with weight function W. We use of the following ingredients:

- A collision-resistant hash family given by an algorithm  $\mathcal{H}$ .
- A SNARG  $(G_{\text{SNARG}}, P_{\text{SNARG}}, V_{\text{SNARG}})$  for  $\hat{\mathcal{L}}$ .
- A rate-1 functional BARG with strong somewhere extraction  $(G_{\text{BARG}}, P_{\text{BARG}}, V_{\text{BARG}}, E_{\text{BARG}})$ .

By Claim A.1 it is sufficient to construct a mergeable SNARG without a linear verifier. The construction is as follows.

**G:** Given as input the security parameter  $\lambda$  the generation algorithm is as follows:

- Sample a hash  $h \leftarrow \mathcal{H}(\lambda)$ .
- Samples a SNARG key  $\sigma \leftarrow G_{\text{SNARG}}(\lambda)$ .
- Let  $\mathcal{F}_\perp$  be a machine that always outputs  $\perp$  padded to size  $\bar{F}(\lambda)$  where  $\bar{F}(\lambda)$  is a polynomial specified in the analysis.
- For  $j \in [2]$ , samples BARG keys  $(\beta_j, \tau_j) \leftarrow G_{\text{BARG}}(\lambda, \mathcal{F}_\perp)$ .
- Output  $(h, \sigma, \beta_1, \beta_2)$ .

To describe the algorithms  $(P, V, PM)$  will first define the “core” algorithms  $(\hat{P}, \hat{V}, \hat{P}\hat{M})$  that operate on extended statements of the form  $\hat{x} = (n \in \mathbb{N}^+, d \in \{0, 1\}^\lambda, x)$  and are given a version of the public key that only contains one of the keys  $\beta_1, \beta_2$ . Then we define  $(P, V, PM)$  as wrappers around these core algorithms. We also define an instance merger for extended statements  $\hat{IM}_h$  that given as input  $\hat{x}_1, \dots, \hat{x}_k$  where  $\hat{x}_i = (n_i, d_i, x_i)$  outputs:

$$\hat{IM}_h(\hat{x}_1, \dots, \hat{x}_k) = \left( \sum_{\kappa \in [k]} n_\kappa, h((\hat{x}_1, \dots, \hat{x}_k)), IM(x_1, \dots, x_k) \right) .$$

**$\hat{P}$ :** Given as input the public key pk and  $\hat{x}$  the core prover algorithm is as follows:

- Parse pk as  $(h, \sigma, \beta)$ .

- Parse  $\hat{x}$  as  $(n, d, x)$ .
- Output  $(\emptyset, P_{\text{SNARG}}(\sigma, x))$ .

$\hat{V}$ : Given as input the public key  $\mathbf{pk}$ ,  $\hat{x}$  and a proof  $\Pi$  the core verifier algorithm is as follows:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta)$ .
- Parse  $\hat{x}$  as  $(n, d, x)$ .
- Parse  $\Pi$  as  $(z, \tilde{\Pi})$ .
- If  $n \notin [W(x)]$  then output 0.
- If  $z = \emptyset$ :
  - If  $n \neq 1$  or  $d \neq \perp$  then output 0.
  - Output  $V_{\text{SNARG}}(\sigma, x, \tilde{\Pi})$ .
- Parse  $z$  as  $(\hat{x}_1, \dots, \hat{x}_k)$ .
- If  $k < 2$  then output 0.
- If  $\hat{x} \neq \hat{M}_h(\hat{x}_1, \dots, \hat{x}_k)$  then output 0.
- Let  $\mathcal{M}[\mathbf{pk}, z]$  be a machine that given  $(\kappa, \Pi_\kappa)$  outputs  $\hat{V}(\mathbf{pk}, \hat{x}_\kappa, \Pi_\kappa)$ .
- Output  $V_{\text{BARG}}(\beta, z, (\mathcal{M}[\mathbf{pk}, z], k), \tilde{\Pi})$ .

$\hat{PM}$ : Given as input the public key  $\mathbf{pk}$  and pairs  $(\hat{x}_1, \Pi_1), \dots, (\hat{x}_k, \Pi_k)$  the core proof merger algorithm is as follows:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta)$ .
- Set  $z \leftarrow (\hat{x}_1, \dots, \hat{x}_k)$ .
- Let  $\mathcal{M}[\mathbf{pk}, z]$  be a machine that given  $(\kappa, \Pi_\kappa)$  outputs  $\hat{V}(\mathbf{pk}, \hat{x}_\kappa, \Pi_\kappa)$ .
- Output  $(z, P_{\text{BARG}}(\beta, z, (\mathcal{M}[\mathbf{pk}, z], k), (\Pi_1, \dots, \Pi_k)))$ .

We proceed to describe the wrapper algorithms  $(P, V, PM)$ .

$P$ : Given as input the public key  $\mathbf{pk}$  and  $x$  the prover algorithm is as follows:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- For  $j \in [2]$ , set  $\mathbf{pk}_j \leftarrow (h, \sigma, \beta_j)$ .
- Let  $n = 1$  and  $d = \perp$ .
- For  $j \in [2]$ , set  $\Pi_j \leftarrow \hat{P}(\mathbf{pk}_j, (n, d, x))$ .
- Pad  $\Pi_1$  or  $\Pi_2$  so that  $|\Pi_1| = |\Pi_2|$ .
- Output  $(n, d, \Pi_1, \Pi_2)$ .

$V$ : Given as input the public key  $\mathbf{pk}$ ,  $x$  and a proof  $\Pi$  the verifier algorithm is as follows:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- Parse  $\Pi$  as  $(n, d, \Pi_1, \Pi_2)$ .
- if  $|\Pi_1| \neq |\Pi_2|$  then output 0.

- For  $j \in [2]$ , set  $\mathbf{pk}_j \leftarrow (h, \sigma, \beta_j)$
- Output  $\bigwedge_{j \in [2]} \hat{\mathbf{V}}(\mathbf{pk}_j, (n, d, x), \Pi_j)$ .

**PM:** Given as input the public key  $\mathbf{pk}$  and pairs  $(x_1, \Pi_1), \dots, (x_k, \Pi_k)$  the proof merger algorithm is as follows:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- Parse  $\Pi_\kappa$  as  $(n_\kappa, d_\kappa, \Pi_{1,\kappa}, \Pi_{2,\kappa})$ .
- For  $j \in [2]$ , set  $\mathbf{pk}_j \leftarrow (h, \sigma, \beta_j)$
- Set  $\hat{x}_i \leftarrow (n_i, d_i, x_i)$ .
- Set  $(n, d, x) \leftarrow \hat{\mathbf{IM}}_h(\hat{x}_1, \dots, \hat{x}_k)$ .
- For  $j \in [2]$ , set  $\Pi_j \leftarrow \hat{\mathbf{PM}}(\mathbf{pk}, (\hat{x}_1, \Pi_{j,1}), \dots, (\hat{x}_k, \Pi_{j,k}))$ .
- Pad  $\Pi_1$  or  $\Pi_2$  so that  $|\Pi_1| = |\Pi_2|$ .
- Output  $(n, d, \Pi_1, \Pi_2)$ .

## 6 Analysis

In this section we analyze the construction in Section 5. See Section 1.2 for an overview.

### 6.1 Completeness.

Completeness follow directly from the completeness of the underlying SNARG. To argue merge completeness, fix  $\mathbf{pk}$  and  $(x_1, \Pi_1), \dots, (x_k, \Pi_k)$  such that  $\mathbf{V}(\mathbf{pk}, x_\kappa, \Pi_\kappa) = 1$  for every  $\kappa \in [k]$ , and  $\mathbf{IM}(x_1, \dots, x_k) = x \neq \perp$ . We show that  $\mathbf{V}(\mathbf{pk}, x, \Pi) = 1$  for  $\Pi = \mathbf{PM}(\mathbf{pk}, (x_1, \Pi_1), \dots, (x_k, \Pi_k))$ . To this end:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- Parse  $\Pi_\kappa$  as  $(n_\kappa, d_\kappa, \Pi_{1,\kappa}, \Pi_{2,\kappa})$ .
- Parse  $\Pi$  as  $(n, d, \Pi_1, \Pi_2)$ .
- Parse  $\Pi_j$  as  $(z_j, \tilde{\Pi}_j)$ .
- Set  $\mathbf{pk}_j \leftarrow (h, \sigma, \beta_j)$ .
- Set  $\hat{x}_\kappa \leftarrow (n_\kappa, d_\kappa, x_\kappa)$ .

By construction of  $\mathbf{PM}$  and  $\hat{\mathbf{PM}}$  we have that for  $j \in [2]$ :

$$\begin{aligned} (n, d, x) &= \hat{\mathbf{IM}}_h(\hat{x}_1, \dots, \hat{x}_k) \ , \\ z_j &= (\hat{x}_1, \dots, \hat{x}_k) \ , \\ \tilde{\Pi}_j &= \mathbf{P}_{\text{BARG}}(\beta_j, z_j, (\mathcal{M}[\mathbf{pk}_j, z_j], k), (\Pi_{j,1}, \dots, \Pi_{j,k})) \ . \end{aligned}$$

Therefore, it is sufficient to show that  $n \in [\mathbf{W}(x)]$  and  $\mathbf{V}_{\text{BARG}}(\beta_j, z_j, (\mathcal{M}[\mathbf{pk}_j, z_j], k), \tilde{\Pi}_j) = 1$  for  $j \in [2]$ .

Since  $(n, d, x) = \hat{\text{IM}}_h(\hat{x}_1, \dots, \hat{x}_k)$  we have that  $n = \sum n_\kappa$ . Since  $V(\sigma, x_\kappa, \Pi_\kappa) = 1$  also  $\hat{V}(\text{pk}_j, \hat{x}_\kappa, \Pi_{j,\kappa}) = 1$  and, therefore,  $n_\kappa \in [W(x_\kappa)]$ . Since  $\text{IM}$  is tree-bounded we have that:

$$n = \sum n_\kappa \leq \sum W(x_\kappa) \leq W(x) .$$

Recall that the machine  $\mathcal{M}[\text{pk}_j, z_j]$  given  $(\kappa, \Pi_\kappa)$  outputs  $\hat{V}(\text{pk}_j, \hat{x}_\kappa, \Pi_\kappa)$ . Below we argue that  $\mathcal{M}[\text{pk}_j, z_j]$  runs in time  $t = \text{poly}(\lambda, |\hat{x}_\kappa|, |\Pi_\kappa|)$ . Since  $\hat{V}(\text{pk}_j, \hat{x}_\kappa, \Pi_{j,\kappa}) = 1$  for every  $\kappa \in [k]$ , we have that:

$$((\mathcal{M}[\text{pk}_j, z_j], k), (\Pi_{j,1}, \dots, \Pi_{j,k})) \in \mathcal{R}_B^t .$$

Therefore, by the completeness of the BARG,  $V_{\text{BARG}}(\beta_j, z_j, (\mathcal{M}[\text{pk}_j, z_j], k), \tilde{\Pi}_j) = 1$ .

## 6.2 Efficiency.

Since we set  $\mathcal{F}(\lambda) = \text{poly}(\lambda)$  below, it follows from the efficiency of the underlying SNARG and the efficiency of the BARG that in the completeness experiment, the running time of  $\mathbf{G}$  is  $\text{poly}(\lambda)$ , the running time of  $\mathbf{P}$  is  $\text{poly}(\lambda, T(x))$  and  $|\Pi| = \text{poly}(\lambda)$ . Next, we bound the running time of  $V(\text{pk}, x, \Pi)$  in the soundness experiment. To this end:

- Parse  $\text{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- Parse  $\Pi$  as  $(n, d, \Pi_1, \Pi_2)$ .
- Parse  $\Pi_j$  as  $(z_j, \tilde{\Pi}_j)$ .
- Set  $\text{pk}_j \leftarrow (h, \sigma, \beta_j)$ .
- Set  $\hat{x} \leftarrow (n, d, x)$ .

We first bound the running time of  $\hat{V}(\text{pk}_j, \hat{x}, \Pi_j)$ . We have that  $W(\hat{x})$  runs in time  $\text{poly}(|\hat{x}|)$ . If  $z_j = \emptyset$ , then  $V_{\text{SNARG}}(\sigma, x, \tilde{\Pi}_j)$  runs in time  $\text{poly}(\lambda, |x|, |\Pi_j|)$  by the efficiency of the underlying SNARG. If  $z_j \neq \emptyset$ , parse  $z_j$  as  $(\hat{x}_1, \dots, \hat{x}_k)$ . We have that  $\hat{\text{IM}}_h(\hat{x}_1, \dots, \hat{x}_k)$  runs in time  $\text{poly}(|z_j|)$  and  $V_{\text{BARG}}(\beta_j, z_j, (\mathcal{M}[\text{pk}_j, z_j], k), \tilde{\Pi}_j)$  runs in time  $(O(|z_j|) + |\tilde{\Pi}_j|) \cdot \text{poly}(\lambda)$  by the efficiency of the BARG. Overall, the running time of  $\hat{V}(\text{pk}_j, \hat{x}, \Pi_j)$  is  $\text{poly}(\lambda, |\hat{x}|, |\Pi_j|)$ . Therefore, the running time of  $V(\text{pk}, x, \Pi)$  is  $\text{poly}(\lambda, |x|, |\Pi|)$ . A SNARG with a linear time verifier follows by Claim A.1.

It remains to argue that  $\text{PM}$  runs in polynomial time and that merge efficiency holds. Fix  $\text{pk}$  and  $(x_1, \Pi_1), \dots, (x_k, \Pi_k)$  such that  $V(\text{pk}, x_\kappa, \Pi_\kappa) = 1$  for every  $\kappa \in [k]$ .

- Parse  $\text{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- Parse  $\Pi_\kappa$  as  $(n_\kappa, d_\kappa, \Pi_{1,\kappa}, \Pi_{2,\kappa})$ .
- For  $j \in [2]$ , set  $\text{pk}_j \leftarrow (h, \sigma, \beta_j)$ .
- Set  $\hat{x}_i \leftarrow (n_i, d_i, x_i)$ .
- Set  $z \leftarrow (\hat{x}_1, \dots, \hat{x}_k)$ .
- Set  $w_j \leftarrow (\Pi_{j,1}, \dots, \Pi_{j,k})$ .

We have that  $\hat{\mathbf{M}}_h(\hat{x}_1, \dots, \hat{x}_k)$  runs in time  $\text{poly}(|z|)$ . Since  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_\kappa, \Pi_{j,\kappa})$  runs in time  $\text{poly}(\lambda, |\hat{x}_\kappa|, |\Pi_{j,\kappa}|)$  we have that  $\mathcal{M}[\mathbf{pk}_j, z]$  runs in time  $\text{poly}(\lambda, |z|, |w_j|)$ . Therefore, by efficiency of efficiency of the BARG and since the BARG has rate 1,  $\mathbf{P}_{\text{BARG}}(\beta, z, (\mathcal{M}[\mathbf{pk}_j, z], k), w_j)$  runs in time  $\text{poly}(\lambda, |z|, |w_j|)$  and outputs a proof of size  $\max |\Pi_{j,\kappa}| + |z| \cdot \text{poly}(\lambda)$ . Since  $\mathbf{V}(\mathbf{pk}, x_\kappa, \Pi_\kappa) = 1$  we have that  $\Pi_{1,\kappa} = \Pi_{2,\kappa} \Rightarrow \max |\Pi_{1,\kappa}| = \max |\Pi_{2,\kappa}|$ . Overall, the running time of  $\mathbf{PM}(\mathbf{pk}, (x_1, \Pi_1), \dots, (x_k, \Pi_k))$  is  $\text{poly}(\lambda, |z|, |(\Pi_1, \dots, \Pi_k)|)$  and it outputs a proof of size:

$$\max |\Pi_{1,\kappa}| + \max |\Pi_{2,\kappa}| + |z| \cdot \text{poly}(\lambda) = \max |\Pi_\kappa| + |z| \cdot \text{poly}(\lambda) .$$

### 6.3 Soundness.

For  $\lambda \in \mathbb{N}$ , let  $\text{Ex}$  denote the soundness experiment:

$$\begin{aligned} \mathbf{pk} &\leftarrow \mathbf{G}(\lambda) \\ (x, \Pi) &\leftarrow \mathbf{Adv}(\mathbf{pk}) \end{aligned} .$$

Assume towards contradiction that there exists a  $\text{poly}(\lambda)$ -size adversary  $\mathbf{Adv}$ , polynomial  $\bar{\mathbf{T}}$  and  $\epsilon(\lambda) = 1/\text{poly}(\lambda)$  such that for infinitely many  $\lambda \in \mathbb{N}$ ,

$$\Pr_{\text{Ex}}[\text{BR}] \geq \epsilon(\lambda) .$$

where  $\text{BR}$  is the event that:

$$x \notin \mathcal{L} \quad \wedge \quad \mathbf{T}(x) \leq \bar{\mathbf{T}}(\lambda) \quad \wedge \quad \mathbf{V}(\mathbf{pk}, x, \Pi) = 1 .$$

**The experiment  $\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}$ .** For a pair of machines  $\mathcal{F}_1, \mathcal{F}_2$  let  $\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}$  be the experiment that is defined just like the soundness experiment  $\text{Ex}$ , except that in the key generation algorithm  $\mathbf{G}$ , the BARG keys  $(\beta_j, \tau_j)$  are sampled from  $\mathbf{G}_{\text{BARG}}(\lambda, \mathcal{F}_j)$  instead of  $\mathbf{G}_{\text{BARG}}(\lambda, \mathcal{F}_\perp)$ .

Since the event  $\text{BR}$  can be tested without the BARG secret keys  $\tau_1, \tau_2$  it follows by the BARG's key-indistinguishability requirement that for every  $\lambda \in \mathbb{N}$  and pair of functions  $\mathcal{F}_1, \mathcal{F}_2$  such that  $|\mathcal{F}_1| = |\mathcal{F}_2| = \bar{\mathbf{F}}(\lambda)$ :

$$\Pr_{\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}}[\text{BR}] \geq \epsilon(\lambda) - \text{negl}(\lambda) . \tag{2}$$

In the experiment  $\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}$  let  $\mathbf{L}_j$  be the list computed by the following procedure:

- Parse  $\mathbf{pk}$  as  $(h, \sigma, \beta_1, \beta_2)$ .
- Parse  $\Pi$  as  $(n_j^{(1)}, d_j^{(1)}, \Pi_1^{(1)}, \Pi_2^{(1)})$ .
- Set  $\mathbf{pk}_j \leftarrow (h, \sigma, \beta_j)$ .
- Set  $\hat{x}_j^{(1)} \leftarrow (n_j^{(1)}, d_j^{(1)}, x)$ .
- Set  $\mathbf{L}_j = []$ .
- Set  $i = 1$ .
- While  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$ :

- Parse  $\Pi_j^{(i)}$  as  $(z_j^{(i)}, \tilde{\Pi}_j^{(i)})$ .
- Append the pair  $(\hat{x}_j^{(i)}, z_j^{(i)})$  to  $\mathbf{L}_j$ .
- If  $z_j^{(i)} = \emptyset$  then break.
- Parse  $z_j^{(i)}$  as  $(\hat{x}_{j,1}^{(i)}, \dots, \hat{x}_{j,k}^{(i)})$
- Set  $\hat{x}_j^{(i+1)} \leftarrow \hat{x}_{j, \mathcal{F}_j(z_j^{(i)})}^{(i)}$ .
- Set  $\Pi_j^{(i+1)} \leftarrow \mathbf{E}_{\text{BARG}}(\tau_j, \tilde{\Pi}_j^{(i)})$ .
- Set  $i \leftarrow i + 1$ .

In what follows we also parse  $\hat{x}_j^{(i)}$  as  $(n_j^{(i)}, d_j^{(i)}, x_j^{(i)})$  and parse  $\hat{x}_{j,\kappa}^{(i)}$  as  $(n_{j,\kappa}^{(i)}, d_{j,\kappa}^{(i)}, x_{j,\kappa}^{(i)})$ .

Since IM is tree-bounded, there exists a polynomial  $Q$  such that  $\mathbf{W}(x^{(1)}) \leq Q(\mathbf{T}(x^{(1)}))$ . Let  $\bar{\mathbf{W}}(\lambda) = Q(\bar{\mathbf{T}}(\lambda))$ . Let LG be the event that BR occurs, and the following conditions hold:

$$\begin{aligned}
& \forall j \in [2], \quad 1 \leq |\mathbf{L}_j| \leq \bar{\mathbf{W}}(\lambda) \quad , \\
& \forall j \in [2], \quad z_j^{(|\mathbf{L}_j|)} = \emptyset \quad , \\
& \forall j \in [2], i \in [|\mathbf{L}_j|] \quad \mathbf{T}(x_j^{(i)}) \leq \bar{\mathbf{T}}(\lambda) \quad , \\
& \forall i \in [\min(|\mathbf{L}_1|, |\mathbf{L}_2|)], \quad (\hat{x}_1^{(i)} = \hat{x}_2^{(i)}) \quad \Rightarrow \quad (z_1^{(i)} = z_2^{(i)}) \quad .
\end{aligned}$$

Note that for LG to occur, it must be that  $|\mathbf{L}_j| \leq \bar{\mathbf{W}}(\lambda)$ . In particular, LG can be tested in time  $\text{poly}(\lambda)$ .

**Claim 6.1.** *For every  $\lambda \in \mathbb{N}$  and pair of functions  $\mathcal{F}_1, \mathcal{F}_2$  such that  $|\mathcal{F}_1| = |\mathcal{F}_2| = \bar{\mathbf{F}}(\lambda)$ :*

$$\Pr_{\mathbf{Ex}_{\mathcal{F}_1, \mathcal{F}_2}} [\text{BR} \Leftrightarrow \text{LG}] \geq 1 - \text{negl}(\lambda) \quad .$$

*Proof of Claim 6.1.* If LG occurs then BR also occurs. It remains to show that if BR occurs then LG also occurs with all but negligible probability.

Fix  $j \in [2]$ . We first argue that  $1 \leq |\mathbf{L}_j| \leq \bar{\mathbf{W}}(\lambda)$ . If BR occurs we have that  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(1)}, \Pi_j^{(1)}) = 1$  and, therefore,  $1 \leq |\mathbf{L}_j|$ . Moreover, if BR occurs then  $\mathbf{T}(x^{(1)}) \leq \bar{\mathbf{T}}(\lambda)$  and since  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(1)}, \Pi_j^{(1)}) = 1$  we have that  $n_j^{(1)} \leq \mathbf{W}(x_j^{(1)}) \leq \bar{\mathbf{W}}(\lambda)$ . Additionally, for every  $i \in [|\mathbf{L}_j|]$  we have that  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  and, therefore,  $n_j^{(i)} > 0$ . Therefore, to argue that  $|\mathbf{L}_j| \leq \bar{\mathbf{W}}(\lambda)$  it is sufficient to show that  $n_j^{(i)} > n_j^{(i+1)}$  for every  $i < |\mathbf{L}_j|$ . Fix  $i < |\mathbf{L}_j|$ . Since  $i \neq |\mathbf{L}_j|$  we have that  $z_j^{(i)} \neq \emptyset$ . Since  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  and  $z_j^{(i)} \neq \emptyset$  it follows that  $\hat{x}_j^{(i)} = \mathbf{IM}_h(\hat{x}_{j,1}^{(i)}, \dots, \hat{x}_{j,k}^{(i)})$  and  $k \geq 2$ . Therefore, we have that  $n_j^{(i)} = \sum n_{j,\kappa}^{(i)}$  which implies that  $n_j^{(i)} > n_{j,\kappa}^{(i)}$  for every  $\kappa \in [k]$ . Since  $\hat{x}_j^{(i+1)} = \hat{x}_{j,\kappa}^{(i)}$  for  $\kappa = \mathcal{F}_j(z_j^{(i)})$  it follows that  $n_j^{(i)} > n_j^{(i+1)}$ .

Next, we argue that  $\mathbf{T}(x_j^{(i)}) \leq \bar{\mathbf{T}}(\lambda)$  for every  $i \in [|\mathbf{L}_j|]$ . If BR occurs we have that  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(1)}, \Pi_j^{(1)}) = 1$  and, therefore,  $1 \leq |\mathbf{L}_j|$ . Moreover, if BR occurs then  $\mathbf{T}(x^{(1)}) \leq \bar{\mathbf{T}}(\lambda)$ . It is sufficient to show that  $\mathbf{T}(x_j^{(i+1)}) \leq \mathbf{T}(x_j^{(i)})$  for every  $i < |\mathbf{L}_j|$ . Fix  $i < |\mathbf{L}_j|$ . Since  $i \neq |\mathbf{L}_j|$  we have that  $z_j^{(i)} \neq \emptyset$ . Since  $\hat{\mathbf{V}}(\mathbf{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  and  $z_j^{(i)} \neq \emptyset$  it follows that  $\hat{x}_j^{(i)} = \mathbf{IM}_h(\hat{x}_{j,1}^{(i)}, \dots, \hat{x}_{j,k}^{(i)})$ .

Therefore, we have that  $x_j^{(i)} = \text{IM}(x_{j,1}^{(i)}, \dots, x_{j,k}^{(i)})$  which implies that  $\mathsf{T}(x_{j,\kappa}^{(i)}) \leq \mathsf{T}(x_j^{(i)})$  for every  $\kappa \in [k]$ . Since  $x_j^{(i+1)} = x_{j,\kappa}^{(i)}$  for  $\kappa = \mathcal{F}_j(z_j^{(i)})$  it follows that  $\mathsf{T}(x_j^{(i+1)}) \leq \mathsf{T}(x_j^{(i)})$ .

Next, we argue that  $z_j^{(i)} = \emptyset$  for  $i = |\mathsf{L}_j|$ . If BR occurs we have that  $1 \leq i$ . If  $z_j^{(i)} \neq \emptyset$  then  $\hat{\mathsf{V}}(\mathsf{pk}_j, \hat{x}_j^{(i+1)}, \Pi_j^{(i+1)}) = 0$  since Otherwise,  $(\hat{x}_j^{(i+1)}, z_j^{(i+1)})$  would have been added to  $\mathsf{L}_j$ . For  $\kappa = \mathcal{F}_j(z_j^{(i)})$  we have that  $\hat{x}_j^{(i+1)} = \hat{x}_{j,\kappa}^{(i)}$  and  $\Pi_j^{(i+1)} = \mathsf{E}_{\text{BARG}}(\tau_j, \tilde{\Pi}_j^{(i)})$ . Since  $\hat{\mathsf{V}}(\mathsf{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  also:

$$\mathsf{V}_{\text{BARG}}(\beta_j, z_j^{(i)}, (\mathcal{M}[\mathsf{pk}_j, z_j^{(i)}], k), \tilde{\Pi}_j^{(i)}) = 1 .$$

However,  $\mathcal{M}[\mathsf{pk}_j, z_j^{(i)}]$  given  $(\kappa, \Pi_j^{(i+1)})$  outputs:

$$\hat{\mathsf{V}}(\mathsf{pk}_j, \hat{x}_{j,\kappa}^{(i)}, \Pi_j^{(i+1)}) = \hat{\mathsf{V}}(\mathsf{pk}_j, \hat{x}_j^{(i+1)}, \Pi_j^{(i+1)}) = 0 .$$

Let  $\text{Adv}'$  be an adversary that given the BARG keys  $(\beta_j, \tau_j) \leftarrow \mathsf{G}_{\text{BARG}}(\lambda, \mathcal{F}_j)$ , emulates the experiment  $\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}$  and outputs  $(z_j^{(i)}, \mathcal{M}[\mathsf{pk}_j, z_j^{(i)}], \tilde{\Pi}_j^{(i)})$ . We have that if BR occurs and  $z_j^{(i)} \neq \emptyset$  with non-negligible probability then  $\text{Adv}'$  breaks the BARG's strong somewhere-extraction requirement. It follows that  $z_j^{(i)} = \emptyset$  with all but negligible probability.

Finally, we argue that for every  $i \in [\min(|\mathsf{L}_1|, |\mathsf{L}_2|)]$ , if  $\hat{x}_1^{(i)} = \hat{x}_2^{(i)}$  then  $z_1^{(i)} = z_2^{(i)}$ . Fix any  $i \in [\min(|\mathsf{L}_1|, |\mathsf{L}_2|)]$  such that  $\hat{x}_1^{(i)} = \hat{x}_2^{(i)}$ . For  $j \in [2]$  since  $\hat{\mathsf{V}}(\mathsf{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  for  $j \in [2]$  it follows that either  $z_j^{(i)} = \emptyset$  and  $d_j^{(i)} = \perp$ , or  $z_j^{(i)} \neq \emptyset$  and  $\hat{x}^{(i)} = \hat{\text{IM}}_h(\hat{x}_{j,1}^{(i)}, \dots, \hat{x}_{j,k}^{(i)})$  which implies that  $d_j^{(i)} = h(z_j^{(i)})$ . Since  $\hat{x}_1^{(i)} = \hat{x}_2^{(i)}$  also  $d_1^{(i)} = d_2^{(i)}$ . Therefore, either  $z_1^{(i)} = z_2^{(i)} = \emptyset$ , or  $h(z_1^{(i)}) = h(z_2^{(i)})$ . Let  $\text{Adv}'$  be an adversary that given  $h \leftarrow \mathcal{H}(\lambda)$ , emulates the experiment  $\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}$  and outputs  $(z_1^{(i)}, z_2^{(i)})$ . We have that if  $\hat{x}_1^{(i)} = \hat{x}_2^{(i)}$  and  $z_1^{(i)} \neq z_2^{(i)}$  with non-negligible probability then  $\text{Adv}'$  breaks the collision resistance of the hash family. It follows that  $\hat{x}_1^{(i)} = \hat{x}_2^{(i)}$  implies  $z_1^{(i)} = z_2^{(i)}$  with all but negligible probability.  $\square$

**The experiment  $\text{Ex}_{f_1, f_2}$ .** For  $\lambda \in \mathbb{N}$  and  $n \leq 2^\lambda$  let  $\Upsilon_n$  be a randomized procedure that outputs an element that is  $2^{-\lambda}$  close to uniform in  $[n]$  in polynomial time  $Q_\Upsilon(\lambda)$ . For any function  $f : \mathbb{N} \rightarrow \{0, 1\}^{Q_\Upsilon(\lambda)} \cup \{\perp\}$  (given by a sufficiently small machine), we denote by  $\mathcal{F}[f]$  that takes as input  $z$  and proceeds as follows:

- Parse  $z$  as  $(\hat{x}_1, \dots, \hat{x}_k)$ .
- Parse  $\hat{x}_\kappa$  as  $(n_\kappa, d_\kappa, x_\kappa)$ .
- Set  $n = \sum_{\kappa \in [k]} n_\kappa$ .
- Set  $\rho \leftarrow f(n)$ .
- If  $\rho = \perp$  then output  $\perp$ .
- Sample  $n_\rho \leftarrow \Upsilon_n$  using randomness  $\rho$ .
- Output the minimal index  $\kappa \in [k]$  such that  $n_\rho \leq \sum_{\kappa' \in [\kappa]} n_{\kappa'}$ .

Set  $\bar{F}(\lambda) = \text{poly}(\lambda)$  to be a polynomial such that  $|\mathcal{F}[f]| \leq \bar{F}(\lambda)$  for any  $\lambda \in \mathbb{N}$  and for any of the functions  $f$  used in the analysis below. We pad the machine  $\mathcal{F}[f]$  so its size is exactly  $\bar{F}(\lambda)$ .

For a pair of functions  $f_1, f_2$  let  $\text{Ex}_{f_1, f_2}$  be the experiment  $\text{Ex}_{\mathcal{F}_1, \mathcal{F}_2}$  where  $\mathcal{F}_j = \mathcal{F}[f_j]$ . Recall that conditioned on  $\text{LG}$  occurring, we have that  $|\mathcal{L}_j| > 0$ . Moreover,  $z_j^{(|\mathcal{L}_j|)} = \emptyset$  and, since  $\hat{V}(\text{pk}_j, \hat{x}_j^{(|\mathcal{L}_j|)}, \Pi_j^{(|\mathcal{L}_j|)}) = 1$ , this implies that  $n_j^{(|\mathcal{L}_j|)} = 1$ . For  $n \in \mathbb{N}$  we denote by  $\mathcal{L}_j^n$  the prefix of  $\mathcal{L}_j$  of length  $i$  for the minimal index  $i$  such that  $n_j^{(i)} \leq n$ . If  $\text{LG}$  occurs such an index  $i$  must exist. If  $\text{LG}$  does not occur then we set  $\mathcal{L}_j^n$  to be empty.

For a function  $f$  we denote by  $f(> n)$  the restriction of  $f$  to inputs  $n' > n$ .

**Claim 6.2.** *For every  $\lambda \in \mathbb{N}$ ,  $n \in \mathbb{N}$  and a pair of functions  $f_1, f_2$  such that  $f_1(> n) = f_2(> n)$  in the experiment  $\text{Ex}_{f_1, f_2}$  it holds that  $\mathcal{L}_1^n = \mathcal{L}_2^n$ .*

*Proof.* If  $\text{LG}$  does not occur then  $\mathcal{L}_1^n$  and  $\mathcal{L}_2^n$  are both empty. Assuming  $\text{LG}$  occurs, we prove that  $\mathcal{L}_1^n = \mathcal{L}_2^n$  by induction on the length of their common prefix. Since  $\text{BR}$  occurs we have that  $\hat{V}(\text{pk}_j, \hat{x}_j^{(1)}, \Pi_j^{(1)}) = 1$  and, therefore,  $1 \leq \min(|\mathcal{L}_1|, |\mathcal{L}_2|)$ . By definition we have that  $\hat{x}_1^{(1)} = \hat{x}_2^{(1)}$  and since  $\text{LG}$  occurs, we have that also  $z_1^{(1)} = z_2^{(1)}$ . If  $n_1^{(1)} = n_2^{(1)} > n$  we have that  $|\mathcal{L}_1^n| = |\mathcal{L}_2^n| = 0$ . Otherwise,  $1 \leq \min(|\mathcal{L}_1^n|, |\mathcal{L}_2^n|)$ .

Assume that for some  $i \leq \min(|\mathcal{L}_1^n|, |\mathcal{L}_2^n|)$  we have that  $(\hat{x}_1^{(i)}, z_1^{(i)}) = (\hat{x}_2^{(i)}, z_2^{(i)})$ . We show that either  $|\mathcal{L}_1^n| = |\mathcal{L}_2^n| = i$  or that the same holds for  $i+1$ . If  $n_1^{(i)} = n_2^{(i)} \leq n$  we have that  $|\mathcal{L}_1^n| = |\mathcal{L}_2^n| = i$ . Moreover, if  $z_1^{(i)} = z_2^{(i)} = \emptyset$  then  $|\mathcal{L}_1| = |\mathcal{L}_2| = i$  and therefore also  $|\mathcal{L}_1^n| = |\mathcal{L}_2^n| = i$ . Otherwise, since  $z_1^{(i)} = z_2^{(i)} \neq \emptyset$  and  $\text{LG}$  occurs, we have that  $i+1 \leq \min(|\mathcal{L}_1|, |\mathcal{L}_2|)$ . Moreover, since  $n_1^{(i)} = n_2^{(i)} > n$  we have that  $i+1 \leq \min(|\mathcal{L}_1^n|, |\mathcal{L}_2^n|)$ .

For  $j \in [2]$  we have that  $\hat{V}(\text{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$ , which implies that  $n_j^{(i)} = \sum n_{j, \kappa}^{(i)}$ . Recall that since  $z_1^{(i)} = z_2^{(i)}$ , the evaluations of  $\mathcal{F}_1(z_1^{(i)})$  and  $\mathcal{F}_2(z_2^{(i)})$  only differ in that  $\mathcal{F}_j$  evaluates the underlying function  $f_j$  on  $n_j^{(i)} = \sum n_{j, \kappa}^{(i)}$ . Since  $n_j^{(i)} \geq n$  and  $f_1(\geq n) = f_2(\geq n)$  we have that  $\mathcal{F}_1(z_1^{(i)}) = \mathcal{F}_2(z_2^{(i)})$ . Since we set  $\hat{x}_j^{(i+1)}$  to  $\hat{x}_j^{(i)}$  we have that  $\hat{x}_1^{(i+1)} = \hat{x}_2^{(i+1)}$ . Since  $\text{LG}$  occurs, we have

that also  $z_1^{(i+1)} = z_2^{(i+1)}$ . □

**The experiment  $\text{Ex}_f$ .** For a function  $f$ , let  $\text{Ex}_f$  be the experiment  $\text{Ex}_{f, f_\perp}$  where  $f_\perp$  is the constant  $\perp$  function.

**Claim 6.3.** *For every polynomial-size distinguisher  $\mathcal{D}$ ,  $\lambda \in \mathbb{N}$ ,  $n \in \mathbb{N}$  and a pair of functions  $f_1, f_2$  such that  $f_1(> n) = f_2(> n)$ :*

$$\left| \Pr_{\text{Ex}_{f_1}} [\text{LG} \wedge \mathcal{D}(\mathcal{L}_1^n) = 1] - \Pr_{\text{Ex}_{f_2}} [\text{LG} \wedge \mathcal{D}(\mathcal{L}_1^n) = 1] \right| \leq \text{negl}(\lambda) .$$

*Proof.* Fix  $j \in [2]$ . By Claim 6.1:

$$\left| \Pr_{\text{Ex}_{f_j}} [\text{LG} \wedge \mathcal{D}(\mathcal{L}_1^n) = 1] - \Pr_{\text{Ex}_{f_j}} [\text{BR} \wedge \mathcal{D}(\mathcal{L}_1^n) = 1] \right| \leq \text{negl}(\lambda) .$$

Since the event BR can be tested and  $L_1^n$  can be computed without the BARG secret key  $\tau_2$  it follows by the BARG's key-indistinguishability requirement that:

$$\left| \Pr_{\text{Ex}_{f_j}} [\text{BR} \wedge \mathcal{D}(L_1^n) = 1] - \Pr_{\text{Ex}_{f_j, f_1}} [\text{BR} \wedge \mathcal{D}(L_1^n) = 1] \right| \leq \text{negl}(\lambda) .$$

By Claim 6.1:

$$\left| \Pr_{\text{Ex}_{f_j, f_1}} [\text{BR} \wedge \mathcal{D}(L_1^n) = 1] - \Pr_{\text{Ex}_{f_j, f_1}} [\text{LG} \wedge \mathcal{D}(L_1^n) = 1] \right| \leq \text{negl}(\lambda) .$$

By Claim 6.2:

$$\left| \Pr_{\text{Ex}_{f_j, f_1}} [\text{LG} \wedge \mathcal{D}(L_1^n) = 1] - \Pr_{\text{Ex}_{f_j, f_1}} [\text{LG} \wedge \mathcal{D}(L_2^n) = 1] \right| \leq \text{negl}(\lambda) .$$

By Claim 6.1:

$$\left| \Pr_{\text{Ex}_{f_j, f_1}} [\text{LG} \wedge \mathcal{D}(L_2^n) = 1] - \Pr_{\text{Ex}_{f_j, f_1}} [\text{BR} \wedge \mathcal{D}(L_2^n) = 1] \right| \leq \text{negl}(\lambda) .$$

Since the event BR can be tested and  $L_2^n$  can be computed without the BARG secret key  $\tau_1$  it follows by the BARG's key-indistinguishability requirement that:

$$\left| \Pr_{\text{Ex}_{f_j, f_1}} [\text{BR} \wedge \mathcal{D}(L_2^n) = 1] - \Pr_{\text{Ex}_{f_1, f_1}} [\text{BR} \wedge \mathcal{D}(L_2^n) = 1] \right| \leq \text{negl}(\lambda) .$$

Combining the above inequalities gives that for every  $j \in [2]$ :

$$\left| \Pr_{\text{Ex}_{f_j}} [\text{LG} \wedge \mathcal{D}(L_1^n) = 1] - \Pr_{\text{Ex}_{f_1, f_1}} [\text{BR} \wedge \mathcal{D}(L_2^n) = 1] \right| \leq \text{negl}(\lambda) .$$

The claim follows.  $\square$

**Main claim.** In what follows we focus only on  $L_1$ , suppressing the index  $j = 1$ . For  $n \in \mathbb{N}$ , let  $i^n = |L^n|$ . We define the random variables  $\alpha^n$  as follows: If  $i^n = 0$  or if  $x^{(i^n)} \in \mathcal{L}$  then  $\alpha^n = 0$ . Otherwise  $\alpha^n = 1/n^{(i^n)}$ .

Let  $f$  be a prefix-puncturable PRF with output length  $Q_{\text{r}}(\lambda)$ .

**Claim 6.4.** For every  $\lambda \in \mathbb{N}$  and  $n \in [\bar{W}(\lambda)]$  and for a random PRF key  $s \leftarrow \{0, 1\}^\lambda$ :

$$\mathbb{E}_{\text{Ex}_{f_{s[n]}}} [\alpha^n] \geq \frac{\epsilon(\lambda)}{\bar{W}(\lambda)} - \text{negl}(\lambda) .$$

Before proving Claim 6.4 we use it to conclude the soundness proof. For  $n = 1$ , Claim 6.4 states that:

$$\mathbb{E}_{\text{Ex}_{f_{s[1]}}} [\alpha^1] \geq \frac{\epsilon(\lambda)}{\bar{W}(\lambda)} - \text{negl}(\lambda) .$$

Let  $i = i^1$ . Since  $n^{(i)} \geq 1$  it follows that  $\alpha^1 \leq 1$ . Therefore:

$$\Pr_{\text{Ex}_{f_s[1]}} [\alpha^1 = 1] \geq \frac{\epsilon(\lambda)}{\bar{W}(\lambda)} - \text{negl}(\lambda) .$$

If  $\alpha^1 \neq 0$  then LG occurs and  $x^{(i)} \notin \mathcal{L}$ . Since LG occurs we have that  $\mathsf{T}(x^{(i)}) \leq \bar{\mathsf{T}}(\lambda)$ . We argue that  $z_j^{(i)} = \emptyset$ . If that is not the case then, since  $\hat{\mathsf{V}}(\text{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  also  $\hat{x}^{(i)} = \hat{\mathsf{M}}_h(\hat{x}_{j,1}^{(i)}, \dots, \hat{x}_{j,k}^{(i)})$  and  $k \geq 2$ . Therefore,  $n_j^{(i)} = \sum n_{j,\kappa}^{(i)}$ , contradicting the fact that  $n^{(i)} \leq 1$ . Since  $z_j^{(i)} = \emptyset$  and  $\hat{\mathsf{V}}(\text{pk}_j, \hat{x}_j^{(i)}, \Pi_j^{(i)}) = 1$  we have that  $\mathsf{V}_{\text{SNARG}}(\sigma, x^{(i)}, \tilde{\Pi}^{(i)}) = 1$ . Let  $\text{Adv}'$  be an adversary that given the SNARG key  $\sigma \leftarrow \mathsf{G}_{\text{SNARG}}(\lambda)$ , emulates the experiment  $\text{Ex}_{f_s[1]}$  and outputs  $(x^{(i)}, \tilde{\Pi}^{(i)})$ . We have that for every  $\lambda \in \mathbb{N}$ :

$$\Pr \left[ \begin{array}{l} \sigma \leftarrow \mathsf{G}(\lambda) \\ (x^{(i)}, \tilde{\Pi}^{(i)}) \leftarrow \text{Adv}'(\sigma) \end{array} : \begin{array}{l} x^{(i)} \notin \mathcal{L} \\ \mathsf{T}(x^{(i)}) \leq \bar{\mathsf{T}}(\lambda) \\ \mathsf{V}_{\text{SNARG}}(\sigma, x^{(i)}, \tilde{\Pi}^{(i)}) = 1 \end{array} \right] \geq \Pr_{\text{Ex}_{f_s[1]}} [\alpha^1 = 1] \geq \frac{\epsilon(\lambda)}{\bar{W}(\lambda)} - \text{negl}(\lambda) ,$$

contradicting the soundness requirement of the underlying SNARG. This concludes the soundness proof.

*Proof of Claim 6.4.* It is sufficient to show that the claim holds for  $n = \bar{W}(\lambda)$  and that for every  $1 < n \leq \bar{W}(\lambda)$ :

$$\mathbb{E}_{\text{Ex}_{f_s[n]}} [\alpha^n] - \mathbb{E}_{\text{Ex}_{f_s[n-1]}} [\alpha^{n-1}] \leq \text{negl}(\lambda) . \quad (3)$$

For  $n = \bar{W}(\lambda)$  we have that if LG occurs then:

$$\hat{\mathsf{V}}(\text{pk}, \hat{x}^{(1)}, \Pi^{(1)}) = 1 \wedge \mathsf{T}(x^{(1)}) \leq \bar{\mathsf{T}}(\lambda) \Rightarrow n^{(1)} \leq \mathsf{W}(x^{(1)}) \leq \bar{W}(\lambda) \Rightarrow i^{\bar{W}(\lambda)} = 1 .$$

Moreover:

$$\hat{x}^{(1)} \notin \mathcal{L} \Rightarrow \alpha^{\bar{W}(\lambda)} = \frac{1}{n^{(1)}} \geq \frac{1}{\bar{W}(\lambda)} .$$

By Claim 6.1 and Eq. (2) we have that for every  $\lambda \in \mathbb{N}$ :

$$\mathbb{E}_{\text{Ex}_{f_s[\bar{W}(\lambda)]}} [\alpha^{\bar{W}(\lambda)}] \geq \mathbb{E}_{\text{Ex}_{f_s[\bar{W}(\lambda)]}} [\alpha^{\bar{W}(\lambda)} \mid \text{LG}] \cdot \Pr_{\text{Ex}_{f_s[\bar{W}(\lambda)]}} [\text{LG}] \geq \frac{\epsilon(\lambda)}{\bar{W}(\lambda)} - \text{negl}(\lambda) .$$

We proceed to prove Eq. (3). For every  $\lambda \in \mathbb{N}$  fix any  $1 < n \leq \bar{W}(\lambda)$  and let  $i = i^n$ .

Let  $\rho \leftarrow \{0, 1\}^{Q_{\mathsf{r}}(\lambda)}$  be a random string and let  $f_{s[n], \rho}$  denote the function  $f_{s[n]}$  where the output on  $n$  is changed from  $\perp$  to  $\rho$ . Let  $\mathcal{F}_{s[n], \rho} = \mathcal{F}[f_{s[n], \rho}]$ .

We define the random variable  $\alpha$  as follows: If  $i = 0$  or if  $n^{(i)} < n$  then  $\alpha = \alpha^{n-1}$ . Otherwise:

- Set  $\kappa \leftarrow \mathcal{F}_{s[n], \rho}(z^{(i)})$
- If  $x_\kappa^{(i)} \in \mathcal{L}$  then set  $\alpha = 0$ .
- Otherwise, set  $\alpha = 1/n_\kappa^{(i)}$ .

**Claim 6.5.** For every  $\lambda \in \mathbb{N}$  and  $1 < n \leq \bar{W}(\lambda)$ :

$$\mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha^n - \alpha] \leq \text{negl}(\lambda) .$$

Before proving Claim 6.5 we use it to conclude the proof of Claim 6.4.

First we argue that,  $\alpha^n$  and  $\alpha$  can be computed in polynomial time given  $L_1^n$ . If  $i = 0$  then  $\alpha^n = \alpha = 0$ . Otherwise, LG occurs and we have that  $T(x_j^{(i)}) \leq \bar{T}(\lambda)$ . Therefore, we can decide if  $x^{(i)} \in \mathcal{L}$  and compute  $\alpha^n$  in polynomial time. Since  $\hat{V}(\text{pk}, \hat{x}^{(i)}, \Pi^{(i)}) = 1$  we have that  $x^{(i)} = \text{IM}(x_1^{(i)}, \dots, x_k^{(i)})$  which implies that  $T(x_\kappa^{(i)}) \leq T(x^{(i)})$ . Therefore, we can decide if  $x_\kappa^{(i)} \in \mathcal{L}$  and compute  $\alpha$  in polynomial time.

Since  $f_{s[n]}(> n) = f_{s[n], \rho}(> n)$  we have that by Claim 6.3, for every  $\alpha'$  in the support of  $\alpha^n$  and  $\alpha$ :

$$\begin{aligned} \left| \Pr_{\rho, \text{Ex}_{f_{s[n]}}} [\text{FS} \wedge \alpha^n = \alpha'] - \Pr_{\text{Ex}_{f_{s[n], \rho}}} [\text{FS} \wedge \alpha^n = \alpha'] \right| &\leq \text{negl}(\lambda) , \\ \left| \Pr_{\rho, \text{Ex}_{f_{s[n]}}} [\text{FS} \wedge \alpha = \alpha'] - \Pr_{\text{Ex}_{f_{s[n], \rho}}} [\text{FS} \wedge \alpha = \alpha'] \right| &\leq \text{negl}(\lambda) . \end{aligned}$$

Since  $\alpha^n$  and  $\alpha$  can only take  $n + 1$  different values and  $n \leq \bar{W}(\lambda)$  it follows that:

$$\left| \mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha^n - \alpha] - \mathbb{E}_{\text{Ex}_{f_{s[n], \rho}}}[\alpha^n - \alpha] \right| \leq \text{negl}(\lambda) .$$

Observe that in the experiment  $\text{Ex}_{f_{s[n], \rho}}$  either  $n^{(i)} < n$  in which case we set  $\alpha = \alpha^{n-1}$ , or  $n^{(i)} = n$  in which case  $\alpha$  and  $\alpha^{n-1}$  are computed in exactly the same way. In any case we have that  $\alpha = \alpha^{n-1}$ . That is:

$$\mathbb{E}_{\text{Ex}_{f_{s[n], \rho}}}[\alpha^n - \alpha] = \mathbb{E}_{\text{Ex}_{f_{s[n], \rho}}}[\alpha^n - \alpha^{n-1}] .$$

Let  $\rho' = f_s(n)$ . By the pseudorandomness of  $f$  we have that:

$$\left| \mathbb{E}_{\text{Ex}_{f_{s[n], \rho}}}[\alpha^n - \alpha^{n-1}] - \mathbb{E}_{\text{Ex}_{f_{s[n], \rho'}}}[\alpha^n - \alpha^{n-1}] \right| \leq \text{negl}(\lambda) .$$

Since  $f_{s[n], \rho'}(> n - 1) = f_{s[n-1]}(> n - 1)$  we have that by Claim 6.3:

$$\left| \mathbb{E}_{\text{Ex}_{f_{s[n], \rho'}}}[\alpha^n - \alpha^{n-1}] - \mathbb{E}_{\text{Ex}_{f_{s[n-1]}}}[\alpha^n - \alpha^{n-1}] \right| \leq \text{negl}(\lambda) .$$

Claim 6.4 follows by combining Claim 6.5 with equality and inequalities above. It remains to prove Claim 6.5.

*Proof of Claim 6.5.* Let FS be the event that LG occurs,  $n^{(i)} = n$  and  $x^{(i)} \notin \mathcal{L}$ . If FS does not occur then:

- Either LG does not occur. In this case  $i = i^{n-1} = 0$  and, therefore,  $\alpha^n = \alpha^{n-1} = \alpha = 0$ .
- Or, LG occurs, but  $n^{(i)} \neq n$ . In this case  $i \neq 0$  and, by the definition of  $L^n$ , we have  $n^{(i)} \leq n$ . Therefore,  $n^{(i)} \leq n - 1$  which implies that  $i = i^{n-1}$  and  $\alpha^n = \alpha^{n-1} = \alpha$ .

- Or, LG occurs and  $n^{(i)} = n$ , but  $x^{(i)} \in \mathcal{L}$ . In this case  $\alpha^n = 0$  and, therefore,  $\alpha^n \leq \alpha$ .

Overall, we have that if FS does not occur then  $\alpha^n \leq \alpha$ . Therefore:

$$\mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha^n - \alpha] \leq \mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha^n - \alpha \mid \text{FS}] \cdot \Pr_{\rho, \text{Ex}_{f_{s[n]}}}[\text{FS}] .$$

If the RHS is negative then we are done. Otherwise, we have that:

$$\mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha^n - \alpha] \leq \mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha^n - \alpha \mid \text{FS}] .$$

If FS occurs then  $\alpha^n = 1/n$ . Therefore, it is sufficient to prove that:

$$\mathbb{E}_{\rho, \text{Ex}_{f_{s[n]}}}[\alpha \mid \text{FS}] \geq \frac{1}{n} - \text{negl}(\lambda) . \quad (4)$$

Fix any randomness of the experiment  $\text{Ex}_{f_{s[n]}}$  such that the event FS occurs. Since FS occurs, we have that  $x^{(i)} \notin \mathcal{L}$ . Since  $\hat{V}(\text{pk}, \hat{x}^{(i)}, \Pi^{(i)}) = 1$  we have that  $x^{(i)} = \text{IM}_h(x_1^{(i)}, \dots, x_k^{(i)})$  and, therefore, there exists  $\bar{\kappa} \in [k]$  such that  $x_{\bar{\kappa}}^{(i)} \notin \mathcal{L}$ . Conditioned on  $\kappa = \bar{\kappa}$  we have that  $\alpha = 1/n_{\bar{\kappa}}^{(i)}$ . Therefore, since  $\alpha \geq 0$ :

$$\mathbb{E}_{\rho}[\alpha] \geq \frac{1}{n_{\bar{\kappa}}^{(i)}} \cdot \Pr_{\rho}[\kappa = \bar{\kappa}] .$$

Since FS occurs, we have that  $n^{(i)} = n$ . Recall that the value  $\kappa = \mathcal{F}_{s[n], \rho}(z^{(i)})$  is computed as follows:

- Set  $n' = \sum_{\kappa' \in [k]} n_{\kappa'}^{(i)} = n^{(i)} = n$ .
- Set  $\rho' \leftarrow f_{s[n], \rho}(n') = f_{s[n], \rho}(n) = \rho$ .
- Sample  $n_{\rho} \leftarrow \Upsilon_n$  using randomness  $\rho' = \rho$ .
- Output the minimal index  $\kappa' \in [k]$  such that  $n_{\rho} \leq \sum_{\kappa'' \in [\kappa']} n_{\kappa''}^{(i)}$ .

Since  $\rho$  is uniform we have that  $n_{\rho}$  is  $2^{-\lambda}$  close to uniform in  $[n]$  and, therefore:

$$\Pr_{\rho}[\kappa = \bar{\kappa} \mid \text{FS}] \geq \frac{n_{\bar{\kappa}}^{(i)}}{n} - \text{negl}(\lambda) .$$

Since  $n_{\bar{\kappa}}^{(i)} \leq n^{(i)} \leq \bar{W}(\lambda)$  It follows that:

$$\mathbb{E}_{\rho}[\alpha] \geq \frac{1}{n_{\bar{\kappa}}^{(i)}} \cdot \Pr_{\rho}[\kappa = \bar{\kappa}] \geq \frac{1}{n} - \text{negl}(\lambda) .$$

Since this is true for every randomness of the experiment  $\text{Ex}_{f_{s[n]}}$  such that FS occurs, Eq. (4) and Claim 6.5 follow. □

□

## A SNARGs with a Linear Verifier

**Claim A.1.** *Assuming one-way functions and BARGs, if there exists a mergeable SNARG for  $\mathcal{L}$  then there exists a mergeable SNARG for  $\mathcal{L}$  with a linear verifier.*

*Proof sketch.* The high-level idea is to compose the original proof system with a RAM SNARG. This idea was previously used in different setting [CJJ21, BBK<sup>+</sup>23]. We sketch the transformation and explain how to apply it in the setting of mergeable SNARGs.

In RAM SNARG for  $\mathcal{P}$ , the verifier is given as input a short digest of the statement  $x$  instead of the statement itself and its running time is  $\text{poly}(\lambda)$ . The soundness condition is that no poly-size cheating prover given a random public key can produce a false statement  $x$  together with a proof, such that the verifier accepts the proof given the digest of  $x$ . We can turn any RAM SNARK into a standard SNARG by having the verifier hash the statement  $x$  itself. This new SNARG has the property that the verifier runs in time  $|x| \cdot \text{poly}(\lambda)$  (dominated by the time required to compute the digest of  $x$ ).

Using such SNARG with linear verifier, we can turn any mergeable SNARG for  $\mathcal{L}$  into a mergeable SNARG for  $\mathcal{L}$  with a linear verifier by “pushing” the work of the verifier to the prover/proof merger. For an instance  $x$ , the proof of the new mergeable SNARG consist of the proof  $\Pi$  of the original mergeable SNARG and of another proof using the SNARG with linear verifier, for the fact that the verifier of the original mergeable SNARG accepts the proof  $\Pi$ . The verification of the new mergeable SNARG simply runs the verifier of SNARG with linear verifier on the input that contains  $x$  and  $\Pi$ . Since the new proof contains the original proof  $\Pi$  we can merge proofs by using the proof merger of the original mergeable SNARG. The proof merger of the new mergeable SNARG simply adds to this merged proof, another proof using the SNARG with linear verifier for the fact that the verifier of the original mergeable SNARG accepts the merged proof. The result is a mergeable SNARG with a linear verifier, where we add  $\text{poly}(\lambda, |x|, |\Pi|)$  to the running time of the new prover and proof merger (the time to construct a proof the execution of the original mergeable SNARG verifier).  $\square$

## References

- [BBK<sup>+</sup>23] Zvika Brakerski, Maya Farber Brodsky, Yael Tauman Kalai, Alex Lombardi, and Omer Paneth. Snargs for monotone policy batch NP. *IACR Cryptol. ePrint Arch.*, page 1050, 2023.
- [BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKS and proof-carrying data. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *Symposium on Theory of Computing Conference, STOC’13, Palo Alto, CA, USA, June 1-4, 2013*, pages 111–120. ACM, 2013.
- [BCJP24] Maya Farber Brodsky, Arka Rai Choudhuri, Abhishek Jain, and Omer Paneth. Monotone-policy aggregate signatures. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part IV*, volume 14654 of *Lecture Notes in Computer Science*, pages 168–195. Springer, 2024.
- [BCMS20] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. Recursive proof composition from accumulation schemes. In Rafael Pass and Krzysztof Pietrzak, editors, *Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2020.
- [BCPR16] Nir Bitansky, Ran Canetti, Omer Paneth, and Alon Rosen. On the existence of extractable one-way functions. *SIAM J. Comput.*, 45(5):1910–1952, 2016.
- [BG08] Boaz Barak and Oded Goldreich. Universal arguments and their applications. *SIAM J. Comput.*, 38(5):1661–1694, 2008.
- [BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Halo: Recursive proof composition without a trusted setup. *IACR Cryptol. ePrint Arch.*, page 1021, 2019.
- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In Hugo Krawczyk, editor, *Public-Key Cryptography - PKC 2014 - 17th International Conference on Practice and Theory in Public-Key Cryptography, Buenos Aires, Argentina, March 26-28, 2014. Proceedings*, volume 8383 of *Lecture Notes in Computer Science*, pages 501–519. Springer, 2014.
- [BKP<sup>+</sup>23] Nir Bitansky, Chethan Kamath, Omer Paneth, Ron Rothblum, and Prashant Nalini Vasudevan. Batch proofs are statistically hiding. *Electron. Colloquium Comput. Complex.*, TR23-077, 2023.
- [BP15] Elette Boyle and Rafael Pass. Limits of extractability assumptions with distributional auxiliary input. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29 - December 3, 2015, Proceedings, Part II*, volume 9453 of *Lecture Notes in Computer Science*, pages 236–261. Springer, 2015.

- [BPSS23] Nir Bitansky, Omer Paneth, Dana Shamir, and Tomer Solomon. Non-interactive universal arguments. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 132–158. Springer, 2023.
- [BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. *IACR Cryptol. ePrint Arch.*, page 352, 2013.
- [CGJ<sup>+</sup>23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and snargs from sub-exponential DDH. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 635–668. Springer, 2023.
- [CGSY24] Alessandro Chiesa, Ziyi Guan, Shahar Samocha, and Eylon Yogev. Security bounds for proof-carrying data from straightline extractors. In Elette Boyle and Mohammad Mahmoody, editors, *Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part II*, volume 15365 of *Lecture Notes in Computer Science*, pages 464–496. Springer, 2024.
- [CJJ21] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. Snargs for  $\mathcal{P}$  from LWE. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, pages 68–79. IEEE, 2021.
- [CT10] Alessandro Chiesa and Eran Tromer. Proof-carrying data and hearsay arguments from signature cards. In Andrew Chi-Chih Yao, editor, *Innovations in Computer Science - ICS 2010, Tsinghua University, Beijing, China, January 5-7, 2010. Proceedings*, pages 310–331. Tsinghua University Press, 2010.
- [DGKV22] Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. Rate-1 non-interactive arguments for batch-np and applications. In *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, pages 1057–1068. IEEE, 2022.
- [DMP26] Nico Döttling, Giulio Malavolta, and Omer Paneth. Batch arguments with optimal communication. *IACR Cryptol. ePrint Arch.*, page 3, 2026.
- [GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th Annual Symposium on Foundations of Computer Science, West Palm Beach, Florida, USA, 24-26 October 1984*, pages 464–479. IEEE Computer Society, 1984.
- [GVW02] Oded Goldreich, Salil P. Vadhan, and Avi Wigderson. On interactive proofs with a laconic prover. *Comput. Complex.*, 11(1-2):1–53, 2002.
- [HW14] Pavel Hubáček and Daniel Wichs. On the communication complexity of secure function evaluation with long output. *IACR Cryptol. ePrint Arch.*, page 669, 2014.

- [IKO05] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. Sufficient conditions for collision-resistant hashing. In Joe Kilian, editor, *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings*, volume 3378 of *Lecture Notes in Computer Science*, pages 445–456. Springer, 2005.
- [KLVW23] Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. Boosting batch arguments and RAM delegation. In Barna Saha and Rocco A. Servedio, editors, *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, pages 1545–1552. ACM, 2023.
- [KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *2013 ACM SIGSAC Conference on Computer and Communications Security, CCS’13, Berlin, Germany, November 4-8, 2013*, pages 669–684. ACM, 2013.
- [KRR13] Yael Tauman Kalai, Ran Raz, and Ron D. Rothblum. Delegation for bounded space. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *Symposium on Theory of Computing Conference, STOC’13, Palo Alto, CA, USA, June 1-4, 2013*, pages 565–574. ACM, 2013.
- [KVZ21] Yael Tauman Kalai, Vinod Vaikuntanathan, and Rachel Yun Zhang. Somewhere statistical soundness, post-quantum security, and snargs. In Kobbi Nissim and Brent Waters, editors, *Theory of Cryptography - 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8-11, 2021, Proceedings, Part I*, volume 13042 of *Lecture Notes in Computer Science*, pages 330–368. Springer, 2021.
- [LM91] Luc Longpré and Sarah Mocas. Symmetry of information and one-way functions. In Wen-Lian Hsu and Richard C. T. Lee, editors, *ISA ’91 Algorithms, 2nd International Symposium on Algorithms, Taipei, Republic of China, December 16-18, 1991, Proceedings*, volume 557 of *Lecture Notes in Computer Science*, pages 308–315. Springer, 1991.
- [LP22] Yanyi Liu and Rafael Pass. On one-way functions from np-complete problems. In Shachar Lovett, editor, *37th Computational Complexity Conference, CCC 2022, Philadelphia, PA, USA, July 20-23, 2022*, volume 234 of *LIPICs*, pages 36:1–36:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
- [Mic94] Silvio Micali. CS proofs (extended abstracts). In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, pages 436–453. IEEE Computer Society, 1994.
- [PP22] Omer Paneth and Rafael Pass. Incrementally verifiable computation via rate-1 batch arguments. In *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, pages 1045–1056. IEEE, 2022.
- [Val08] Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *TCC*, pages 1–18, 2008.

- [WW22] Brent Waters and David J. Wu. Batch arguments for snfp and more from standard bilinear group assumptions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 433–463. Springer, 2022.
- [ZL70] A. K. Zvonkin and L. A. Levin. The complexity of finite objects and the development of the concepts of information and randomness by means of the theory of algorithms. *Russian Mathematical Surveys*, 25(6):83–124, 1970.